Mediahub

DIPLOMARBEIT

verfasst im Rahmen der

Reife- und Diplomprüfung

an der

Höheren Abteilung für IT-Medientechnik

Eingereicht von:
Markus Tran
Abdulkerim Cufurovic

Betreuer:
Prof. Mag. Robert Reder

Leonding, 4. April 2026

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt bzw. die wörtlich oder sinngemäß entnommenen Stellen als solche kenntlich gemacht habe.

Die Arbeit wurde bisher in gleicher oder ähnlicher Weise keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Die vorliegende Diplomarbeit ist mit dem elektronisch übermittelten Textdokument identisch.

Leonding, 4. April 2026 Markus Tran & Abdulkerim Cufurovic

Kurzfassung

Die vorliegende Diplomarbeit behandelt die Konzeption und Umsetzung der Webanwendung „Mediahub“, deren Ziel in der zentralen Verwaltung persönlicher Streaming-Abonnements mit Fokus auf Filme liegt. Ausgangspunkt ist die Beobachtung, dass bei parallel genutzten Streaming-Diensten die Übersicht über Kosten, Laufzeiten, Kündigungszeitpunkte und tatsächliche Verfügbarkeiten rasch verloren geht. Daraus ergibt sich ein praktisches Problem, das sowohl organisatorische als auch technische Anforderungen umfasst.

Im Rahmen des Projekts wurde gemeinsam eine Anwendung entwickelt, die Filminformationen, Anbieter-Verfügbarkeiten und persönliche Abo-Daten in einer gemeinsamen Oberfläche zusammenführt. Die Arbeitsteilung im Projekt erfolgte zwischen Frontend- und Backend-Entwicklung. Im Frontend wurden insbesondere die Abo-Verwaltung, die Filmsuche, die Film-Detailansicht, die Darstellung passender Anbieter sowie geschützte Benutzerbereiche auf Basis von Keycloak umgesetzt. Dadurch entstand eine funktionale Webanwendung, die einen konkreten Mehrwert für Nutzende bietet.

Die Arbeit zeigt zugleich, dass die Entwicklung einer solchen Anwendung nicht nur aus der Implementierung einzelner Funktionen besteht. Wesentliche Herausforderungen lagen in der sinnvollen Übersetzung von Backend-Daten in die Benutzeroberfläche, in der Integration von Authentifizierung sowie in der bewussten Begrenzung des Projektumfangs. Nicht alle begonnenen Funktionen wurden vollständig ausgebaut; die Verlaufsfunktion ist beispielsweise nur teilweise umgesetzt. Insgesamt stellt die Arbeit jedoch eine fachlich nachvollziehbare und technisch tragfähige Grundlage für spätere Erweiterungen dar.

Abstract

This diploma thesis presents the design and implementation of the web application „Mediahub“, whose purpose is the central management of personal streaming subscriptions with a focus on films. The starting point is the observation that, when several streaming services are used in parallel, users quickly lose track of costs, billing cycles, cancellation dates, and actual film availability. This creates a practical problem that combines organisational and technical aspects.

Within the project, an application was developed collaboratively that combines film information, provider availability, and personal subscription data in a single interface. The work was divided between frontend and backend development. On the frontend side, the implemented functionality includes subscription management, film search, a film detail view, the presentation of matching providers, and protected user areas based on Keycloak. As a result, a functional web application was created that already provides practical value for its users.

At the same time, the thesis shows that developing such an application involves more than implementing isolated features. Key challenges included translating backend data into a clear user interface, integrating authentication, and deliberately limiting the project scope. Not every planned feature was completed in full; for example, the history functionality was only implemented partially. Overall, however, the project provides a technically sound and professionally coherent foundation for future extensions.

Inhaltsverzeichnis

Kurzfassung **I**

Abstract **II**

1 Einleitung **1**

1.1 Motivation und Problemstellung der Abo-Verwaltung 1

1.2 Zielsetzung der Diplomarbeit 2

1.3 Nutzen der Anwendung für Endanwender 2

1.4 Aufbau der Diplomarbeit 3

2 Ausgangssituation und Analyse **4**

2.1 Aktuelle Herausforderungen bei Streaming-Abonnements 4

2.2 Probleme durch parallele Abos und doppelte Abbuchungen 4

2.3 Bestehende Lösungen und deren Schwächen 5

2.4 Abgrenzung zu Standard-Webprojekten 6

3 Fachlicher Hintergrund Abo- & Mediatheken-Management **7**

3.1 Zentrale Verwaltung von Abonnements und Laufzeiten 7

3.2 Darstellung von Verfügbarkeiten von Filmen 8

3.3 Mehrwert einer zentralen Applikation für Nutzer 8

3.4 Datenschutz- und Sicherheitsanforderungen 9

4 Technologische Grundlagen **10**

4.1 Angular 10

4.2 Keycloak 15

4.3 Tailwind CSS 16

4.4 GitHub 20

4.5 Quarkus – Grundlagen und Einsatz in der Anwendung 22

4.6 Grundprinzip von Quarkus 22

4.7 Quarkus in der vorliegenden Applikation 24

4.8 Keycloak aus Backend-Sicht 24
4.9 Keycloak und JWT-Authentifizierung 24

5 Architektur und Systemdesign 27
5.1 Gesamtarchitektur der Anwendung 27
5.2 Kommunikation zwischen Frontend und Backend 28
5.3 API-Integration externer Movie-Datenbanken 29
5.4 Sicherheits- und Datenschutzkonzept 30
5.5 Datenbanken 31
5.6 Datenbankkonzept und architektonische Entscheidungen 32
5.7 Backend-Architektur 36
5.8 Implementierung des Backends in der vorliegenden Applikation 38
5.9 Externe APIs und die Entscheidung für die TMDB API 44

6 Umsetzung der Anwendung 48
6.1 Implementierung der Abo-Verwaltung 48
6.2 Darstellung von Laufzeiten und Kündigungszeitpunkten 50
6.3 Verknüpfung von Filmen mit Streaming-Anbietern 51
6.4 Account- und Passwortverwaltung 54
6.5 Herausforderungen und Reflexion der Frontend-Entwicklung 57

7 Projektverlauf und Entscheidungen 60
7.1 Projektplanung und Arbeitsweise 60
7.2 Technische und fachliche Herausforderungen 61
7.3 Entscheidungsfindung im Projektteam 62
7.4 Einbindung von Betreuern und Feedback 63
7.5 Reaktion auf Fragen und Kritik aus Präsentationen 64

8 Ergebnisse und Evaluation 65
8.1 Erfüllung der gesetzten Ziele 65
8.2 Bewertung des Nutzens für Endanwender 67
8.3 Reflexion der eigenen Leistung 68

9 Ausblick 71
9.1 Mögliche Erweiterungen der Anwendung 71
9.2 Zukunftspotenzial der Plattform 72

10 Zusammenfassung 74

Literaturverzeichnis **VI**

Abbildungsverzeichnis **VIII**

1 Einleitung

1.1 Motivation und Problemstellung der Abo-Verwaltung

Die Nutzung digitaler Streaming-Dienste ist in den vergangenen Jahren zu einem festen Bestandteil des Medienalltags geworden. Viele Nutzende verwenden heute nicht nur einen einzelnen Anbieter, sondern kombinieren mehrere Plattformen parallel, um auf unterschiedliche Filmangebote zugreifen zu können. Dadurch steigt jedoch auch die Komplexität der persönlichen Medienverwaltung. Laufende Kosten verteilen sich auf mehrere Dienste, Kündigungs- und Verlängerungszeitpunkte geraten leicht aus dem Blick und es ist oft unklar, welche Inhalte bereits durch bestehende Abonnements abgedeckt sind.

Aus dieser Situation ergibt sich ein konkretes praktisches Problem: Zwar existieren Plattformen zur Anzeige von Filminformationen und Anbieter-Verfügbarkeiten, ebenso wie allgemeine Werkzeuge zur Verwaltung von Zahlungen oder Abonnements, eine zentrale Anwendung zur Verbindung dieser beiden Bereiche ist jedoch deutlich seltener. Genau an dieser Stelle setzt das Projekt „Mediahub“ an. Ziel ist es, Filmrecherche, Anbieterübersicht und persönliche Abo-Verwaltung in einer gemeinsamen Anwendung zusammenzuführen und dadurch einen realen Mehrwert für Nutzende zu schaffen.

Im Rahmen der Projektarbeit wurden dafür sowohl technische als auch konzeptionelle Fragestellungen betrachtet. Dazu zählen unter anderem die Ausrichtung auf eine geeignete Zielgruppe, die Frage nach relevanten Streaming-Anbietern, die Orientierung an bestehenden Benutzeroberflächen aus verwandten Anwendungsbereichen sowie die sichere Verwaltung geschützter Benutzerfunktionen. Die Einleitung bildet damit den Ausgangspunkt für die fachliche und technische Betrachtung der gesamten Diplomarbeit.

1.2 Zielsetzung der Diplomarbeit

Das zentrale Ziel der Diplomarbeit besteht in der Konzeption und Umsetzung einer Webanwendung zur zentralen Verwaltung von Streaming-Abonnements mit Fokus auf Filme. Die Anwendung soll Nutzende dabei unterstützen, ihre aktiven Dienste übersichtlich zu verwalten, Abrechnungsinformationen nachzuvollziehen und gleichzeitig schnell zu erkennen, auf welchen Plattformen ein gesuchter Film verfügbar ist. Dadurch soll nicht nur der organisatorische Aufwand reduziert, sondern auch eine fundiertere Nutzung bestehender Abonnements ermöglicht werden.

Für die praktische Umsetzung wurden mehrere Teilziele verfolgt. Einerseits sollte eine benutzerfreundliche Oberfläche entstehen, die eine klare und verständliche Verwaltung persönlicher Abonnements ermöglicht. Andererseits sollte eine Such- und Detailfunktion für Filme integriert werden, damit Filmdaten, Anbieterinformationen und persönliche Nutzungssituation gemeinsam betrachtet werden können. Darüber hinaus war ein sicheres Authentifizierungs- und Zugriffskonzept erforderlich, um geschützte Bereiche wie die persönliche Abo-Verwaltung datenschutzkonform bereitzustellen.

Die Diplomarbeit verfolgt damit nicht nur das Ziel, eine funktionierende technische Lösung zu entwickeln, sondern auch zu untersuchen, welche fachlichen Anforderungen für eine solche Anwendung relevant sind. Dazu gehören insbesondere Nutzerbedürfnisse, Plattformabdeckung, Benutzerfreundlichkeit, Sicherheitsaspekte und die sinnvolle Integration bestehender externer Dienste wie TMDB und Keycloak.

1.3 Nutzen der Anwendung für Endanwender

Für Endanwender liegt der wesentliche Nutzen von „Mediahub“ in der zentralen Verwaltung aller Abonnements innerhalb einer einzigen Anwendung. Statt verschiedene Streaming-Plattformen und persönliche Zahlungsinformationen getrennt im Blick behalten zu müssen, erhalten Nutzende eine gemeinsame Übersicht über ihre aktiven Dienste und die damit verbundenen Informationen. Dadurch kann schneller erkannt werden, welche Abonnements tatsächlich genutzt werden und an welchen Stellen unnötige oder doppelte Kosten entstehen.

Ein zusätzlicher Nutzen entsteht durch die Verbindung von persönlicher Abo-Verwaltung mit der Filmsuche. Nutzende können nach Filmen suchen, Detailinformationen aufrufen und gleichzeitig sehen, auf welchen Plattformen ein Film verfügbar ist. Besonders

hilfreich ist dabei die Gegenüberstellung von allgemeinen Anbieterinformationen und den bereits gespeicherten eigenen Abonnements. Diese Funktion unterstützt Nutzende dabei, vorhandene Dienste bewusster zu nutzen und Entscheidungen über neue oder bestehende Abonnements besser einzuordnen.

Auch aus Sicht von Benutzerfreundlichkeit und Sicherheit bietet die Anwendung einen konkreten Mehrwert. Geschützte Bereiche werden über Keycloak abgesichert, sodass persönliche Abo-Daten nicht öffentlich zugänglich sind. Gleichzeitig ermöglicht die strukturierte Oberfläche eine verständliche Navigation zwischen öffentlicher Filmsuche und persönlichem Verwaltungsbereich. Die Anwendung richtet sich damit an Nutzende, die ihre Filmnutzung nicht nur konsumieren, sondern auch organisatorisch effizienter verwalten möchten.

1.4 Aufbau der Diplomarbeit

Die vorliegende Diplomarbeit ist so aufgebaut, dass zunächst die fachliche Problemstellung und anschließend die technische Umsetzung der Anwendung schrittweise erläutert werden. Nach der Einleitung werden in Kapitel 2 die Ausgangssituation und die zentralen Herausforderungen bei Streaming-Abonnements beschrieben. Kapitel 3 vertieft den fachlichen Hintergrund und behandelt unter anderem den Nutzen zentraler Abo-Verwaltung, die Bedeutung von Film-Verfügbarkeiten und datenschutzrelevante Anforderungen.

Kapitel 4 stellt die technologischen Grundlagen der Arbeit vor. Dabei werden die eingesetzten Technologien wie Angular, Keycloak, Tailwind CSS, Quarkus und GitHub im Hinblick auf ihre Rolle innerhalb des Projekts erläutert. In Kapitel 5 folgen Architektur und Systemdesign der Anwendung, einschließlich Gesamtarchitektur, API-Integration, Sicherheitskonzept, Datenbankmodell und Backend-Struktur.

Kapitel 6 beschreibt die konkrete Umsetzung der Anwendung mit Fokus auf die Frontend-Funktionen wie Abo-Verwaltung, Filmsuche, Detailansicht und geschützte Bereiche. Anschließend werden in den weiteren Kapiteln Projektverlauf, Ergebnisse, Ausblick und Zusammenfassung dargestellt. Den Abschluss bilden Literaturverzeichnis und Abbildungsverzeichnis.

2 Ausgangssituation und Analyse

2.1 Aktuelle Herausforderungen bei Streaming-Abonnements

Die Nutzung digitaler Streaming-Dienste hat sich in den letzten Jahren stark verändert. Während zu Beginn oft einzelne Plattformen ausreichten, greifen viele Haushalte heute parallel auf mehrere kostenpflichtige Dienste zu. Gleichzeitig steigen Preise, Inhalte wechseln zwischen Plattformen und die Transparenz über tatsächliche Nutzung und laufende Kosten nimmt ab. Aktuelle Marktbeobachtungen zeigen, dass Haushalte mehrere Streaming-Abonnements parallel verwenden und ihre Ausgaben zunehmend bewusster hinterfragen [1]. Damit entsteht ein Problemfeld, das nicht nur mediale Verfügbarkeit, sondern auch organisatorische und finanzielle Aspekte umfasst.

Für Nutzende bedeutet dies, dass der Zugang zu Filmen nicht mehr nur von persönlichem Interesse abhängt, sondern zunehmend von der Frage, welche Plattform aktuell abonniert wurde und zu welchem Preis. Je mehr Dienste gleichzeitig genutzt werden, desto schwieriger wird es, Kosten, Nutzen und tatsächliche Verwendung in einem ausgewogenen Verhältnis zu halten. Genau an dieser Stelle wird deutlich, dass Streaming nicht nur ein Unterhaltungsangebot ist, sondern auch ein Verwaltungsproblem im Alltag erzeugen kann.

2.2 Probleme durch parallele Abos und doppelte Abbuchungen

Ein wesentliches Problem paralleler Streaming-Abonnements ist die fehlende Übersicht über laufende Kosten. Werden mehrere Dienste gleichzeitig verwendet, verteilen sich Abrechnungen über unterschiedliche Zeitpunkte, Preisstufen und Vertragsmodelle. Für Nutzende ist dadurch oft nur schwer erkennbar, welche Gesamtkosten pro Monat tatsächlich entstehen und welche Plattformen im Verhältnis dazu regelmäßig genutzt

werden. Gerade kleine Einzelbeträge wirken zunächst unproblematisch, summieren sich jedoch über längere Zeiträume zu einem merkbaren Kostenfaktor.

Hinzu kommt, dass unnötige oder doppelte Abonnements häufig nicht unmittelbar auffallen. Nutzende behalten Dienste oft länger aktiv, obwohl bestimmte Plattformen über Wochen oder Monate kaum verwendet werden. Zusätzlich kann es vorkommen, dass ähnliche Inhalte auf mehreren Plattformen verfügbar sind und dadurch mehrere Abos bestehen bleiben, obwohl der praktische Mehrwert begrenzt ist. Ohne zentrale Übersicht entsteht somit leicht eine Situation, in der Kosten weiterlaufen, ohne dass ein bewusster Nutzen damit verbunden ist.

Ein weiterer kritischer Punkt betrifft Kündigungsfristen und Abrechnungszyklen. Werden Verlängerungszeitpunkte nicht aktiv beobachtet, verlängern sich Abonnements automatisch und verursachen weitere Abbuchungen. Für Nutzende bedeutet dies nicht nur finanzielle Mehrkosten, sondern auch zusätzlichen organisatorischen Aufwand. Aus fachlicher Sicht zeigt sich hier deutlich, dass eine Lösung benötigt wird, die nicht nur Anbieter auflistet, sondern auch die Verwaltung persönlicher Abo-Informationen strukturiert unterstützt.

Die daraus entstehenden Problemfelder lassen sich insbesondere in folgende Punkte gliedern:

- fehlende Transparenz über monatliche Gesamtkosten,
- parallele Zahlungen für wenig oder gar nicht genutzte Dienste,
- mangelnde Übersicht über Verlängerungs- und Kündigungszeitpunkte,
- fehlender Zusammenhang zwischen aktivem Abo und tatsächlich gesuchten Filmen.

2.3 Bestehende Lösungen und deren Schwächen

Für Teilaspekte dieses Problems existieren bereits unterschiedliche Lösungsansätze. Einerseits gibt es allgemeine Finanz- oder Haushaltsanwendungen, mit denen wiederkehrende Zahlungen dokumentiert werden können. Diese Werkzeuge konzentrieren sich jedoch meist auf Kostenkontrolle und berücksichtigen nicht, auf welcher Plattform konkrete Filme verfügbar sind. Andererseits existieren Dienste und Datenbanken, die Filminformationen und Anbieter-Verfügbarkeiten anzeigen. Solche Angebote helfen bei

der Suche nach Inhalten, enthalten jedoch in der Regel keine persönliche Verwaltung aktiver Abonnements.

Gerade die Trennung dieser beiden Perspektiven stellt eine wesentliche Schwäche bestehender Lösungen dar. Für Nutzende ist nicht nur relevant, ob ein Film irgendwo verfügbar ist, sondern ob er innerhalb der eigenen bereits bezahlten Abonnements abrufbar ist. Ebenso reicht eine reine Kostenübersicht nicht aus, wenn unklar bleibt, welche Dienste im Hinblick auf konkrete Filminhalte überhaupt einen Mehrwert bieten. Eine Anwendung, die beide Ebenen miteinander verbindet, adressiert daher ein real existierendes praktisches Defizit.

2.4 Abgrenzung zu Standard-Webprojekten

Im Unterschied zu einem klassischen Informations- oder Demo-Webprojekt bestand die Herausforderung dieser Arbeit nicht nur darin, Daten darzustellen oder eine Benutzeroberfläche zu gestalten. Vielmehr musste eine Anwendung konzipiert werden, die externe Filmdaten, Anbieterinformationen, persönliche Abonnements und geschützte Benutzerbereiche sinnvoll zusammenführt. Dadurch entsteht ein deutlich höherer fachlicher Anspruch, weil allgemeine Mediendaten und benutzerspezifische Informationen innerhalb einer gemeinsamen Logik verarbeitet werden müssen.

Die Anwendung „Mediahub“ grenzt sich insbesondere dadurch ab, dass sie nicht nur eine Filmsuche oder eine einfache Abo-Liste bereitstellt, sondern die zentrale Verwaltung aller Abonnements in den Mittelpunkt stellt. Ergänzend dazu werden Filmdaten mit Verfügbarkeiten verknüpft, sodass Nutzende einen unmittelbaren Bezug zwischen ihren aktiven Diensten und konkreten Inhalten erhalten. Auch die Integration geschützter Bereiche über Keycloak zeigt, dass es sich nicht um eine reine Frontend-Demo handelt, sondern um eine fachlich strukturierte Anwendung mit klarer Trennung zwischen öffentlichen und persönlichen Daten.

Die fachliche Komplexität des Projekts zeigt sich damit vor allem in drei Bereichen:

- Verknüpfung externer Filmdaten mit benutzerspezifischen Informationen,
- sichere Trennung zwischen öffentlichen und geschützten Funktionen,
- praktische Ausrichtung auf einen realen Alltagsnutzen statt auf reine Demonstrationszwecke.

3 Fachlicher Hintergrund Abo- & Mediatheken-Management

3.1 Zentrale Verwaltung von Abonnements und Laufzeiten

Eine zentrale Verwaltung von Streaming-Abonnements ist vor allem deshalb relevant, weil moderne Mediennutzung häufig auf mehrere Plattformen verteilt ist. Nutzende greifen nicht mehr nur auf einen einzelnen Anbieter zu, sondern kombinieren unterschiedliche Dienste je nach Filmangebot, Preisstruktur oder zeitlich begrenzten Inhalten. Dadurch entstehen mehrere parallele Vertragsverhältnisse, die hinsichtlich Kosten, Laufzeiten und tatsächlicher Nutzung beobachtet werden müssen. Branchenanalysen zeigen, dass Haushalte mehrere Streaming-Abonnements parallel verwenden und ihre Entscheidungen zunehmend anhand von Preis-Leistungs-Verhältnis, Nutzungsintensität und Kündigungsflexibilität treffen [1].

Aus fachlicher Sicht bedeutet zentrale Verwaltung daher mehr als eine bloße Auflistung aktiver Dienste. Relevant ist vor allem die Zusammenführung von Anbietername, Kostenstruktur, Abrechnungszeitpunkt und persönlicher Nutzung in einer gemeinsamen Übersicht. Erst dadurch wird erkennbar, welche Dienste tatsächlich benötigt werden, welche nur selten verwendet werden und an welchen Stellen Einsparungspotenzial besteht. Eine solche Struktur schafft Transparenz und unterstützt Nutzende dabei, Abonnements bewusster zu verwalten.

Eine fachlich sinnvolle zentrale Verwaltung sollte daher mindestens folgende Informationen berücksichtigen:

- aktiver Streaming-Anbieter,
- Status des Abonnements,
- relevante Zeitpunkte wie letzte Abbuchung oder Verlängerung,
- Bezug zwischen aktivem Abo und verfügbarem Filmangebot.

3.2 Darstellung von Verfügbarkeiten von Filmen

Neben der Verwaltung von Abonnements ist für die praktische Nutzung entscheidend, auf welcher Plattform ein bestimmter Film verfügbar ist. Diese Information besitzt für Nutzende einen unmittelbaren Mehrwert, da sie Suchaufwand reduziert und die Entscheidung unterstützt, ob ein bestehendes Abo ausreicht oder ein zusätzlicher Dienst erforderlich wäre. Die Herausforderung besteht dabei darin, allgemeine Filmdaten und plattformspezifische Anbieterinformationen so zusammenzuführen, dass sie in einer Anwendung verständlich dargestellt werden können.

Für die vorliegende Arbeit war dies besonders relevant, da sich das Projekt derzeit auf Filme konzentriert. Die Anzeige von Verfügbarkeiten bezieht sich daher nicht auf beliebige Medientypen, sondern gezielt auf filmbezogene Daten. Als externe Grundlage diente die TMDB API, die strukturierte Informationen zu Filmen, Metadaten und Anbieter-Verfügbarkeiten bereitstellt [2, 3]. Fachlich zeigt sich daran, dass die Anwendung nicht nur eigene Daten verwaltet, sondern auch externe Informationsquellen sinnvoll in den Nutzungskontext der Nutzenden einbindet.

3.3 Mehrwert einer zentralen Applikation für Nutzer

Der eigentliche Mehrwert einer zentralen Applikation entsteht aus der Verbindung zweier Perspektiven: Einerseits wollen Nutzende ihre laufenden Abonnements kontrollieren, andererseits möchten sie schnell erkennen, ob ein gesuchter Film über bereits vorhandene Dienste verfügbar ist. Werden diese beiden Aspekte getrennt betrachtet, entsteht zusätzlicher Aufwand. Werden sie dagegen in einer einzigen Anwendung zusammengeführt, verbessert sich die Nutzbarkeit deutlich.

Eine zentrale Lösung wie „Mediahub“ ermöglicht daher nicht nur die Verwaltung aller Abos, sondern auch die Einordnung dieser Abos im konkreten Nutzungskontext. Nutzende können nachvollziehen, welche Plattformen für ihre Filmnutzung tatsächlich relevant sind und welche Dienste möglicherweise keinen ausreichenden praktischen Mehrwert mehr bieten. Gerade unter Bedingungen steigender Kosten und wachsender Angebotsvielfalt wird eine solche Form der Bündelung organisatorisch und wirtschaftlich sinnvoll [1].

Daraus ergeben sich für Nutzende insbesondere folgende Vorteile:

- schnellere Orientierung bei der Filmsuche,
- bessere Kontrolle über bestehende Abonnements,
- fundiertere Entscheidungen über Beibehaltung oder Kündigung einzelner Dienste,
- geringerer organisatorischer Aufwand im Medienalltag.

3.4 Datenschutz- und Sicherheitsanforderungen

Sobald eine Anwendung persönliche Abo-Daten speichert und geschützte Funktionen bereitstellt, entstehen zusätzliche Anforderungen an Datenschutz und Zugriffsschutz. Dabei ist zwischen öffentlich verfügbaren Filmdaten und benutzerspezifischen Informationen klar zu unterscheiden. Während Filmtitel, Beschreibungen oder Anbieter-Verfügbarkeiten grundsätzlich allgemeine Daten darstellen, zählen gespeicherte Abonnements, Auswahlentscheidungen und persönliche Verwaltungsinformationen zum geschützten Benutzerkontext.

Für die fachliche Konzeption bedeutet dies, dass personenbezogene oder benutzerspezifische Inhalte nicht ohne Authentifizierung zugänglich sein dürfen. Die Anwendung benötigt daher eine klare Trennung zwischen offenen Bereichen und geschützten Funktionen. Genau aus diesem Grund spielt die Authentifizierung über Keycloak im Projekt eine zentrale Rolle: Sie sorgt dafür, dass persönliche Daten nur nach erfolgreicher Anmeldung verarbeitet und angezeigt werden können. Datenschutz ist in diesem Zusammenhang nicht nur eine technische Zusatzanforderung, sondern ein grundlegender Bestandteil der fachlichen Systemlogik.

4 Technologische Grundlagen

4.1 Angular

Im Rahmen meines Projektteils lag der Schwerpunkt auf der Entwicklung des Frontends. Als technologische Grundlage wurde dafür Angular verwendet, da das Framework eine klare Struktur für moderne Webanwendungen bietet und zentrale Funktionen wie Routing, Komponentenarchitektur, Formularverarbeitung und HTTP-Kommunikation bereits integriert [4, 5, 6, 7, 8]. Für die Umsetzung des MedienAbonnementManagers war Angular besonders geeignet, da mehrere zusammenhängende Oberflächenbereiche entwickelt werden mussten, die dennoch in einer einheitlichen Anwendung miteinander verbunden bleiben sollten.

4.1.1 Rolle von Angular innerhalb der Anwendung

Angular übernimmt in der Anwendung die gesamte Darstellung und Interaktion der Benutzeroberfläche. Dazu zählen insbesondere die Startseite, die Filmübersicht, die Detailansicht einzelner Filme, die Navigation innerhalb der Anwendung sowie die Oberfläche zur Verwaltung persönlicher Streaming-Abonnements. Der Fokus meines Projektteils lag darauf, diese Funktionen so umzusetzen, dass sie übersichtlich, wartbar und für Nutzende intuitiv bedienbar sind. Entscheidend war dabei nicht nur die Darstellung einzelner Seiten, sondern die saubere Verbindung mehrerer Ansichten, API-Aufrufe und Eingabemasken innerhalb einer zusammenhängenden Single-Page-Application.

4.1.2 Architektur als Single-Page-Application

Das Frontend wurde als Single-Page-Application umgesetzt. Bei diesem Ansatz bleibt die Anwendung nach dem ersten Laden im Browser aktiv und Inhalte werden dynamisch aktualisiert, ohne dass bei jeder Aktion ein vollständiger Seitenneuaufbau erfolgen muss [9]. Diese Architektur verbessert die Benutzererfahrung deutlich, da Navigationsvorgänge zwischen Startseite, Filmübersicht, Detailansicht und Providerverwaltung flüssig

ablaufen. Gerade für eine interaktive Webanwendung mit mehreren Ansichten stellt dies einen wesentlichen Vorteil dar.

Für dieses Projekt war dabei vor allem der Angular Router relevant. Angular beschreibt den Router als offiziellen Kernbestandteil des Frameworks für Navigation in Single-Page-Applications [5]. Im MedienAbonnementManager konnten dadurch unterschiedliche Ansichten wie Filmübersicht, Detailseite, Abo-Verwaltung sowie geschützte Bereiche über eine gemeinsame Routing-Struktur organisiert werden. Die Routen werden in einer `Routes`-Konfiguration definiert, die eigentliche Ausgabe erfolgt über `RouterOutlet`, und Navigationspfade werden über `RouterLink` in Templates eingebunden [5]. Dadurch lassen sich Ansichtswechsel innerhalb der Anwendung ohne vollständigen Reload umsetzen, was für die Suche, das Wechseln zwischen Filmlisten und Detailansichten sowie den Zugriff auf persönliche Verwaltungsbereiche besonders sinnvoll war.

Zusätzlich war Routing auch für geschützte Bereiche wichtig. Da Teile der Anwendung nur nach Anmeldung sinnvoll genutzt werden können, etwa die persönliche Abo-Verwaltung, ließ sich dieser Bereich über die Routing-Struktur klar vom öffentlichen Teil der Anwendung trennen. Angular nennt neben Routen, Links und Outlets auch Guards als Teil seines Routing-Ansatzes [5]. Das passte gut zur Projektanforderung, öffentliche Filmansichten und nutzerbezogene Verwaltungsfunktionen sauber innerhalb derselben Oberfläche zu kombinieren. Gleichzeitig ermöglichte die Router-Struktur sprechende und fachlich nachvollziehbare Navigationspfade, etwa für Listenansichten, Detailaufrufe einzelner Filme oder den Wechsel in persönliche Verwaltungsbereiche. Dadurch war die Navigation nicht nur technisch sauber, sondern auch aus Sicht der Benutzerführung konsistent.

4.1.3 Begründung der Wahl von Angular gegenüber React

Für dieses Projekt wurde Angular gewählt, weil es ein vollständiges Framework mit klarer Struktur und fest integrierten Kernfunktionen darstellt [4, 10]. Angular beschreibt sich selbst als Plattform für schnelle, zuverlässige und skalierbare Anwendungen und nennt dabei insbesondere Komponenten, Routing, Dependency Injection und Forms als zusammengehörige Bausteine [4]. Genau diese Kombination war für den MedienAbonnementManager sinnvoll: Die Anwendung besteht nicht aus einer einzelnen Maske, sondern aus mehreren Ansichten mit Navigation, Datenzugriff auf Backend-APIs, Authentifizierungsbezug und formularbasierter Verwaltung von Abonnements.

Im Vergleich zu React bot Angular damit für den Frontend-Teil der Diplomarbeit einen Vorteil, weil wichtige Grundlagen bereits offiziell und konsistent im Framework enthalten sind. Routing, Formulare und HTTP-Kommunikation mussten nicht erst über mehrere zusätzliche Bibliotheken zusammengesetzt werden. Dadurch konnte die Frontend-Architektur schneller stabil aufgebaut werden, während gleichzeitig eine klare Trennung zwischen Ansicht, Datenzugriff und Navigation erhalten blieb. Für ein Projekt mit mehreren funktionalen Bereichen und geplanter Erweiterbarkeit war diese Standardisierung besonders hilfreich.

4.1.4 Komponentenstruktur der Anwendung

Angular beschreibt Anwendungen als Baum aus Komponenten [7]. Dieses Konzept war für den MedienAbonnementManager besonders hilfreich, weil sich die Benutzeroberfläche in mehrere eigenständige und fachlich klar abgegrenzte Teile zerlegen ließ. Dazu gehören unter anderem die Navigationsleiste, Filmkarten in Übersichtsansichten, die Detailansicht eines Films, die Oberfläche zur Abo-Verwaltung sowie die Ansichten für Login und Registrierung. Diese Aufteilung ermöglicht eine saubere Trennung einzelner Funktionsbereiche und reduziert Seiteneffekte bei Änderungen.

Gerade bei wiederkehrenden Oberflächenelementen war diese Struktur ein konkreter Vorteil. Filmkarten konnten in Listenansichten mehrfach verwendet werden, ohne dieselbe Darstellungslogik mehrfach umzusetzen. Die Navbar konnte zentral gepflegt werden, obwohl sie für Navigation, Login-Status und Zugriff auf geschützte Bereiche relevant ist. Auch die Trennung zwischen Übersichts- und Detailansichten wurde dadurch klarer, weil jede Komponente einen eng abgegrenzten Verantwortungsbereich erhielt. Angular unterstützt diese Kapselung direkt über seine Komponentenarchitektur; aktuelle Versionen setzen zudem standardmäßig auf standalone components, was den Import und die Wiederverwendung einzelner Bausteine vereinfacht [7].

4.1.5 Data Binding und Services zur Zustandsverwaltung

Ein wesentlicher Bestandteil der Frontend-Entwicklung war die Verknüpfung von Benutzereingaben mit den dargestellten Daten. Angular stellt dafür Mechanismen wie Data Binding und Formularunterstützung bereit [8]. Im Projekt wurden außerdem eigene Services eingesetzt, um Datenzugriffe zu kapseln und die Kommunikation mit den bereitgestellten Schnittstellen zu organisieren. Dadurch bleibt die Oberflächenlogik

übersichtlich und die einzelnen Komponenten konzentrieren sich auf Darstellung und Interaktion.

Für die Kommunikation mit dem Backend war insbesondere `HttpClient` relevant, den Angular offiziell in `@angular/common/http` bereitstellt [6]. Über diese integrierte HTTP-Schnittstelle konnten Filmdaten, Detailinformationen und Anbieterinformationen vom Backend geladen werden. Angular nennt dabei unter anderem typisierte Antworten, vereinfachte Fehlerbehandlung und Interceptors als Vorteile des HTTP-Clients [6]. Für das Projekt war das deshalb sinnvoll, weil API-Aufrufe nicht direkt in den Komponenten verteilt werden mussten, sondern in Services gebündelt werden konnten. Komponenten übernahmen damit vor allem die Darstellung der Daten und die Reaktion auf Benutzereingaben, während Services den eigentlichen Datenzugriff kapselten.

Diese Trennung war besonders wichtig, weil verschiedene Ansichten auf unterschiedliche Datenquellen zugreifen. Die Filmübersicht benötigt andere Informationen als eine Detailansicht oder die Abo-Verwaltung. Durch eigene Services konnten diese Zugriffe strukturiert getrennt werden, ohne dass die Komponenten mit HTTP-Logik überladen wurden. Ebenso konnten unterschiedliche Endpunkte fachlich sauber gebündelt werden, etwa für Filmdaten, Providerinformationen oder benutzerspezifische Abo-Daten. Zusätzlich ist der Einsatz von Interceptors im Angular-Ökosystem für typische Querschnittsaufgaben wie Header-Ergänzungen, Fehlerbehandlung oder Authentifizierungsbezug vorgesehen [6]. Das unterstützt eine saubere Erweiterbarkeit, falls der Datenzugriff in weiteren Projektphasen noch ausgebaut wird.

Für die Wartbarkeit des Projekts war auch relevant, dass `HttpClient` mit klaren Request- und Response-Strukturen arbeitet. Dadurch konnten Datenobjekte im Frontend typisiert verarbeitet werden, was Fehler bei der Weitergabe von API-Daten reduziert. Gerade bei einer Anwendung, die Daten aus mehreren Quellen zusammenführt, ist diese Klarheit wichtig: Film-Metadaten, Verfügbarkeitsinformationen und benutzerspezifische Abonnementdaten folgen nicht exakt derselben Struktur und müssen im Frontend dennoch konsistent weiterverarbeitet werden. Angular bot hier eine gute Grundlage, um diese Unterschiede kontrolliert in Services abzubilden.

4.1.6 Formularlogik in der Abo-Verwaltung

Angular beschreibt Forms als standardisiertes System für Formularteilnahme und Validierung [4, 8]. Genau dieser Punkt war für die Abo-Verwaltung im Projekt relevant.

Beim Anlegen oder Bearbeiten eines Abonnements müssen mehrere zusammengehörige Eingaben verarbeitet werden, darunter ausgewählter Anbieter, Abrechnungszyklus, Start- oder Abrechnungsdatum sowie Preis. Diese Eingaben stehen in direktem Bezug zur fachlichen Logik der Anwendung und müssen konsistent übernommen werden, damit ein Abo korrekt gespeichert und später in Übersichten oder Berechnungen verwendet werden kann.

Die Formularunterstützung von Angular war hier hilfreich, weil Eingabefelder, Zustandsverwaltung und Validierung nach einem einheitlichen Muster umgesetzt werden konnten. Dadurch ließ sich die Eingabemaske für die Abo-Verwaltung strukturiert aufbauen, anstatt Validierungen und Formularzustände manuell für jedes Feld zu behandeln. Für das Projekt bedeutete das konkret mehr Zuverlässigkeit bei Benutzereingaben und eine bessere Wartbarkeit der Maske, insbesondere bei Formularfeldern mit fachlichen Einschränkungen wie Preis, Datum oder auswählbaren Providern.

Darüber hinaus war die Formularlogik auch für die Benutzerführung relevant. In einer Verwaltungsansicht müssen Eingaben nicht nur korrekt gespeichert, sondern auch verständlich dargestellt und bei Fehlern nachvollziehbar rückgemeldet werden. Angular erleichtert diese Kopplung zwischen Eingabefeld, Formularstatus und Ausgabe [8]. Das war für das Projekt besonders nützlich, weil die Abo-Maske keine rein dekorative Oberfläche ist, sondern ein zentraler Bereich, in dem Nutzende ihre eigenen Daten erfassen und aktualisieren.

4.1.7 Projektstruktur und Erweiterbarkeit

Neben den sichtbaren Oberflächenfunktionen war Angular auch aus Sicht der Projektorganisation sinnvoll. Die Angular-Dokumentation nennt die CLI als üblichen Einstiegspunkt für das Erstellen und Verwalten eines Projekts [11]. Eine solche standardisierte Projektstruktur erleichtert die Arbeit mit Komponenten, Services und Konfigurationen, weil zentrale Muster des Frameworks bereits vorgegeben sind. Für die Diplomarbeit war das hilfreich, da sich neue Bestandteile wie Ansichten, Services oder Formularlogik geordnet in die bestehende Struktur einfügen ließen.

Gerade bei einem Projekt, das sowohl Suche, Detailseiten, Authentifizierung als auch persönliche Verwaltung kombiniert, ist diese Struktur ein Vorteil für die langfristige Erweiterung. Neue Funktionen können als zusätzliche Komponenten, Routen oder Services ergänzt werden, ohne den bisherigen Aufbau grundlegend zu verändern. Dadurch

unterstützt Angular nicht nur die aktuelle Umsetzung, sondern auch die fachliche Weiterentwicklung des Projekts.

4.1.8 Projektbezogene Begründung für Angular

Insgesamt war Angular für den MedienAbonnementManager vor allem deshalb sinnvoll, weil das Framework genau die Bausteine zusammenführt, die in dieser Anwendung benötigt wurden: eine klare Komponentenstruktur für mehrere Ansichten, clientseitige SPA-Navigation, integrierte HTTP-Kommunikation, standardisierte Formularunterstützung und eine Architektur, die sich sauber erweitern lässt [4, 5, 6, 7, 8]. Die Wahl von Angular war damit nicht nur technisch begründbar, sondern direkt aus den Anforderungen der Anwendung ableitbar.

Für eine Webanwendung, die Filmrecherche, Detaildarstellung, Benutzeranmeldung und persönliche Abo-Verwaltung in einer Oberfläche vereint, war diese Kombination besonders passend. Angular half dabei, die Anwendung nicht als lose Sammlung einzelner Seiten umzusetzen, sondern als zusammenhängendes Frontend mit klaren Verantwortlichkeiten zwischen Navigation, Darstellung, Datenzugriff und Eingabelogik.

4.2 Keycloak

Für die Anmeldung und Registrierung der Nutzenden wurde im Frontend Keycloak eingebunden. Dadurch musste keine eigene Login-Oberfläche mit selbst implementierter Passwortlogik entwickelt werden. Stattdessen wurde eine etablierte Lösung für Authentifizierung verwendet, die das Frontend mit dem Anmeldesystem verbindet [12, 13]. Aus Sicht meines Projektteils war vor allem die Einbindung in die Benutzeroberfläche und die Steuerung geschützter Frontend-Bereiche relevant.

4.2.1 Grundlagen der Authentifizierung aus Frontend-Sicht

Im Frontend dient die Authentifizierung dazu, zwischen öffentlichen und geschützten Bereichen der Anwendung zu unterscheiden. Öffentliche Inhalte wie die Filmübersicht können allgemein angezeigt werden, während persönliche Bereiche wie die Verwaltung eigener Abonnements nur angemeldeten Nutzenden zugänglich sind. Das Frontend

reagiert damit auf den Authentifizierungsstatus und passt Navigation sowie verfügbare Funktionen entsprechend an.

4.2.2 Ablauf des Login- und Token-Prozesses

Beim Login wird die anfragende Person aus dem Frontend heraus an Keycloak weitergeleitet. Nach erfolgreicher Anmeldung kehrt sie in die Anwendung zurück und kann geschützte Funktionen nutzen [12]. Für das Frontend ist dabei entscheidend, dass der Authentifizierungsstatus erkannt und für weitere Anfragen verwendet wird. Auf diese Weise können geschützte Ansichten nur dann geöffnet werden, wenn eine gültige Anmeldung vorliegt.

4.2.3 Rollen und Zugriff auf geschützte Bereiche

Neben der reinen Anmeldung spielt auch die Zugriffskontrolle auf bestimmte Ansichten eine Rolle. Im Frontend wird geprüft, ob Nutzende auf geschützte Bereiche wie die Providerverwaltung zugreifen dürfen. Dadurch kann die Navigation benutzerfreundlich gestaltet werden, indem nicht erlaubte Seiten entweder verborgen oder nur nach erfolgreicher Anmeldung erreichbar gemacht werden.

4.2.4 Nutzen für Sicherheit und Benutzerfreundlichkeit

Die Verwendung von Keycloak verbessert sowohl die Sicherheit als auch die Benutzerfreundlichkeit. Da keine eigene Passwortverwaltung entwickelt werden musste, konnte das Risiko unsicherer Eigenimplementierungen reduziert werden. Gleichzeitig wirkt der Login-Prozess für Nutzende konsistent, da Registrierung, Anmeldung und Abmeldung zentral über ein spezialisiertes System gesteuert werden.

4.3 Tailwind CSS

Für die Gestaltung der Benutzeroberfläche wurde Tailwind CSS eingesetzt. Tailwind ist ein CSS-Framework, das nach dem Utility-First-Prinzip arbeitet und dadurch eine schnelle und flexible Entwicklung moderner Benutzeroberflächen ermöglicht [14, 15].

Im Frontend des MedienAbonnementManagers wurde Tailwind genutzt, um Layout, Abstände, Farben, Formulare und Kartenansichten effizient zu gestalten.

4.3.1 Utility-First-Ansatz

Der Utility-First-Ansatz bedeutet, dass viele kleine CSS-Klassen direkt in den Templates verwendet werden, um konkrete Gestaltungseigenschaften festzulegen [14, 16]. Dazu zählen unter anderem Abstände, Größen, Farben, Rahmen oder Typografie. Tailwind beschreibt diesen Ansatz als das Kombinieren vieler einzelner, klar abgegrenzter Utility-Klassen direkt im Markup [16]. Für das Projekt hatte dies den Vorteil, dass Oberflächen schnell angepasst und konsistent gestaltet werden konnten, ohne für jede Änderung umfangreiche eigene CSS-Strukturen anlegen zu müssen.

Gerade im Kontext des MedienAbonnementManagers war das hilfreich, weil viele Oberflächenelemente zwar ähnlich aufgebaut sind, sich aber im Detail unterscheiden. Filmkarten, Filterbereiche, Formularfelder, Buttons und Navigationsbereiche benötigen jeweils leicht unterschiedliche Kombinationen aus Abstand, Typografie, Hervorhebung und Anordnung. Mit Tailwind konnten diese Unterschiede direkt am jeweiligen Element gesteuert werden, ohne dafür erst neue semantische CSS-Klassen oder tiefe Stylesheet-Hierarchien aufzubauen. Das beschleunigte die Umsetzung deutlich und machte den Zusammenhang zwischen Struktur und Gestaltung im Template unmittelbar nachvollziehbar.

Tailwind nennt außerdem mehrere praktische Vorteile dieses Ansatzes: Oberflächen lassen sich schneller umsetzen, Änderungen sind sicherer, weil sie zunächst nur das konkrete Element betreffen, und der CSS-Bestand wächst nicht mit jeder neuen Funktion linear weiter [16]. Genau diese Punkte waren im Projekt relevant, weil sich die Oberfläche während der Umsetzung mehrfach weiterentwickelt hat, etwa bei Suchansichten, Detailseiten und der Abo-Verwaltung. Die Gestaltung konnte dadurch zügig angepasst werden, ohne bestehende Bereiche ständig durch globale CSS-Regeln zu gefährden.

4.3.2 Gestaltung der Benutzeroberfläche

Tailwind CSS wurde besonders für interaktive Bereiche wie die Providerverwaltung, Formulare, Suchfelder, Buttons und Kartenansichten eingesetzt. Dadurch konnte eine moderne und übersichtliche Oberfläche umgesetzt werden, die die verschiedenen Funk-

tionsbereiche visuell voneinander trennt. Vor allem in der Abo-Verwaltung war eine klare Darstellung wichtig, damit Nutzende Anbieter, Status und Eingabefelder schnell erfassen können.

Bei der visuellen Gestaltung spielte nicht nur das Aussehen einzelner Komponenten eine Rolle, sondern auch deren Zusammenwirken auf einer Seite. Übersichtsseiten mussten viele Inhalte gleichzeitig transportieren, ohne unruhig zu wirken: Suchleiste, Filtermöglichkeiten, Listen von Filmen, Providerinformationen und Aktionsbereiche. Tailwind erleichterte dieses Zusammenspiel durch sofort verfügbare Utilities für Flexbox, Grid, Abstandssteuerung, Breiten, Höhen, Typografie und visuelle Hervorhebungen [16]. Dadurch ließen sich Kartenlayouts, Formblöcke und Navigationsleisten schnell so strukturieren, dass die Anwendung trotz ihrer Funktionsvielfalt klar lesbar blieb.

Besonders bei Kartenansichten war dies nützlich. Filmdaten müssen kompakt dargestellt werden, gleichzeitig sollen Poster, Titel, Zusatzinformationen und Aktionen optisch sauber zusammenwirken. Anstatt dafür ein eigenes, umfangreiches CSS-System aufzubauen, konnten mit Tailwind bestehende Utilities direkt kombiniert werden. Das vereinfachte die Entwicklung, weil visuelle Anpassungen unmittelbar in den betroffenen Templates vorgenommen werden konnten.

4.3.3 Responsive Design

Ein wichtiger Vorteil von Tailwind im Projekt war die Unterstützung responsiver Oberflächen. Die Tailwind-Dokumentation betont, dass Utility-Klassen über Breakpoints bedingt angewendet werden können und sich dadurch responsive Interfaces direkt im HTML umsetzen lassen [17]. Für den MedienAbonnementManager war das relevant, weil die Anwendung sowohl auf größeren Bildschirmen als auch auf kleineren Geräten sinnvoll nutzbar bleiben sollte.

Gerade Übersichtsseiten mit Filmlisten oder Verwaltungsansichten profitieren davon. Auf breiteren Displays können Inhalte nebeneinander angeordnet werden, während auf kleineren Geräten eine vertikale Struktur oft besser geeignet ist. Mit Tailwind ließ sich dieses Verhalten über responsive Varianten direkt an den betreffenden Elementen steuern [17]. Dadurch mussten für unterschiedliche Bildschirmgrößen nicht separate Stylesheets gepflegt werden. Für das Projekt bedeutete das eine höhere Konsistenz zwischen Desktop- und Mobilansicht bei gleichzeitig geringerem Wartungsaufwand.

4.3.4 Interaktive Zustände und Benutzerführung

Tailwind bietet Varianten für Hover-, Focus-, Active- und weitere Zustände direkt auf Utility-Ebene [18]. Die Dokumentation beschreibt, dass Utility-Klassen durch Varianten wie `hover:`, `focus:` oder `active:` bedingt angewendet werden können [18]. Das war für das Projekt besonders hilfreich, weil die Anwendung stark von Interaktion lebt: Filme auswählen, Buttons auslösen, Eingabefelder verwenden, Navigationspunkte erkennen und geschützte Bereiche bedienen.

In der Praxis verbesserte das die Benutzerführung deutlich. Buttons konnten beim Überfahren visuell reagieren, Eingabefelder beim Fokussieren klar hervorgehoben werden und aktive Navigations- oder Aktionszustände wurden verständlicher. Gerade in Formularen und Verwaltungsansichten ist dieses Feedback wichtig, weil Nutzende dort viele kleine Aktionen hintereinander ausführen. Tailwind ermöglichte diese Interaktionszustände ohne zusätzliche komplexe CSS-Definitionen und blieb dennoch konsistent innerhalb des gesamten Frontends.

4.3.5 Formulare und Eingabemasken

Für die Abo-Verwaltung spielte Tailwind auch bei der Formulargestaltung eine wichtige Rolle. Die Eingabemaske musste mehrere Felder übersichtlich anordnen, visuell gruppieren und in einen gut lesbaren Gesamtaufbau einbetten. Utility-Klassen für Abstand, Größe, Rahmen, Farben, Fokuszustände und Typografie waren dafür besonders nützlich [16, 18]. Dadurch konnten Eingabefelder, Labels, Auswahlmenüs und Aktionsbuttons so gestaltet werden, dass sie als zusammengehörige Maske wahrgenommen werden.

Für das Projekt war das deshalb wichtig, weil Formulare nicht nur funktional, sondern auch verständlich und vertrauenswürdig wirken müssen. Wenn Nutzende Anbieter, Preise oder Datumsangaben eintragen, soll die Maske klar strukturiert sein und Rückmeldungen optisch eindeutig vermitteln. Tailwind unterstützte dies, indem dieselben Gestaltungsprinzipien über die gesamte Abo-Verwaltung hinweg konsistent angewendet werden konnten.

4.3.6 Integration von Tailwind CSS in Angular

Die Einbindung von Tailwind CSS in das Angular-Projekt erfolgte direkt über die Konfiguration des Frontends. Die Tailwind-Dokumentation beschreibt dafür einen offiziellen

Installationsweg für Angular-Projekte, bei dem Tailwind über die Angular-Umgebung und PostCSS eingebunden wird [19]. Dadurch standen die Utility-Klassen in allen Komponenten zur Verfügung. Diese enge Integration war für die praktische Frontend-Entwicklung besonders hilfreich, da Layout und Funktionalität parallel umgesetzt werden konnten.

Im Projekt war das besonders vorteilhaft, weil Angular-Komponenten ihre Templates direkt mit den passenden Tailwind-Klassen versehen konnten. Struktur, Verhalten und Gestaltung blieben dadurch nah beieinander. Wenn sich etwa eine Kartenansicht, ein Suchbereich oder ein Formularfeld verändert hat, konnte die visuelle Anpassung direkt in der betreffenden Komponente erfolgen. Das reduzierte Reibungsverluste zwischen Template- und CSS-Arbeit und passte gut zum komponentenbasierten Aufbau des Frontends.

4.3.7 Beitrag zur Benutzerfreundlichkeit

Ein zentrales Ziel des Frontends war eine intuitive und ansprechende Bedienung. Die Gestaltung mit Tailwind CSS unterstützt dieses Ziel, indem wichtige Elemente klar hervorgehoben und logisch angeordnet werden. Eine gut strukturierte Oberfläche trägt wesentlich dazu bei, dass Nutzende die Anwendung leichter verstehen und effizient verwenden können.

Projektbezogen war Tailwind deshalb nicht nur eine Styling-Hilfe, sondern ein Werkzeug zur strukturierten Umsetzung der gesamten Frontend-Oberfläche. Die Kombination aus Utility-First-Ansatz, responsiven Varianten, Zustandsklassen und guter Angular-Integration passte sehr gut zu einer Anwendung, die Suchoberflächen, Detailansichten, Authentifizierungsbezug und Abo-Verwaltung in einer konsistenten Benutzeroberfläche zusammenführt [16, 17, 18, 19]. Tailwind war damit für dieses Projekt nicht bloß eine schnelle CSS-Lösung, sondern ein sinnvoller Bestandteil der gesamten Frontend-Architektur.

4.4 GitHub

GitHub wurde im Projekt als Plattform für Versionsverwaltung und Zusammenarbeit genutzt. Für meinen Frontend-Teil war dies insbesondere wichtig, um Änderungen

an Komponenten, Styles und Routing nachvollziehbar zu dokumentieren und in einem strukturierten Entwicklungsprozess festzuhalten [20]. Dadurch konnten einzelne Entwicklungsschritte sauber voneinander getrennt und schrittweise weiterentwickelt werden.

4.4.1 Versionsverwaltung im Frontend

Die Frontend-Entwicklung umfasste mehrere aufeinander aufbauende Änderungen, beispielsweise an der Filmsuche, an der Navigationsstruktur, an der Provideransicht und am visuellen Design. Durch die Versionsverwaltung konnten diese Schritte einzeln dokumentiert und bei Bedarf auf frühere Stände zurückgeführt werden. Dies erhöhte die Nachvollziehbarkeit und reduzierte das Risiko, funktionierende Zwischenstände zu verlieren.

4.4.2 Branching und strukturierte Weiterentwicklung

Im Projekt wurden verschiedene Branches verwendet, um neue Änderungen getrennt vom Hauptstand weiterzuentwickeln. Für die Frontend-Arbeit war dies besonders hilfreich, wenn größere Überarbeitungen an der Benutzeroberfläche oder an der Logik einzelner Ansichten vorgenommen wurden. So konnten neue Ideen und Anpassungen zunächst isoliert entwickelt werden, bevor sie in einen stabileren Stand übernommen wurden.

4.4.3 Nachvollziehbarkeit der eigenen Arbeit

Die Commit-Historie dokumentiert die schrittweise Weiterentwicklung des Frontends, etwa bei der Überarbeitung der Filmsuche, beim Ausbau der Trending-Ansichten oder bei der Verbesserung der Benutzeroberfläche der Abo-Verwaltung. Für die Diplomarbeit ist dies hilfreich, da der eigene Beitrag am Frontend dadurch nachvollziehbar beschrieben werden kann.

4.5 Quarkus – Grundlagen und Einsatz in der Anwendung

Quarkus ist ein modernes, quelloffenes Java-Framework, das speziell für den Einsatz in containerisierten und Cloud-nativen Umgebungen entwickelt wurde. Ziel des Frameworks ist es, den Prozess der Bereitstellung von Java-Applikationen in Cloud-Umgebungen zu vereinfachen und zu optimieren. Dabei unterstützt Quarkus Entwicklerinnen und Entwickler dabei, Applikationen auch ohne tiefgehende Kenntnisse der zugrundeliegenden Infrastruktur bereitzustellen.

Ein wesentliches Merkmal von Quarkus ist seine Kompatibilität mit einer Vielzahl etablierter Frameworks und Standards. Dazu zählen unter anderem Eclipse MicroProfile, RESTEasy (JAX-RS), Hibernate ORM (JPA), Apache Kafka sowie Spring. Diese breite Unterstützung ermöglicht es, bestehende Kenntnisse und Bibliotheken direkt einzusetzen, ohne grundlegende Umstrukturierungen vornehmen zu müssen.

Quarkus hebt sich durch seine Leichtgewichtigkeit von anderen Java-Frameworks ab. Konkret äußert sich dies in einer geringen Startzeit, einem niedrigen Speicherverbrauch sowie der einfachen Möglichkeit, Projekte in Cloud-Umgebungen bereitzustellen.

4.6 Grundprinzip von Quarkus

Ein zentrales Merkmal von Quarkus liegt in der Art, wie Java-Anwendungen kompiliert und ausgeführt werden. Um dies zu verstehen, ist es notwendig, die zwei grundlegenden Kompilierungsansätze zu unterscheiden: die Just-In-Time-Kompilierung (JIT) und die Ahead-of-Time-Kompilierung (AOT).

4.6.1 Just-In-Time-Kompilierung (JIT)

Im klassischen Java-Ausführungsmodell wird Quellcode zunächst vom Java-Compiler in plattformunabhängigen Bytecode übersetzt. Dieser Bytecode wird anschließend von der Java Virtual Machine (JVM) interpretiert. Die JIT-Kompilierung ist dabei ein Optimierungsmechanismus der JVM: Häufig ausgeführte Codeabschnitte werden zur Laufzeit in nativen Maschinencode übersetzt, was die Ausführungsgeschwindigkeit erhöht.

Der Nachteil dieses Ansatzes liegt darin, dass die JVM beim Start der Applikation zunächst hochgefahren werden muss und die JIT-Optimierungen erst schrittweise während der Laufzeit greifen. Dies führt zu längeren Startzeiten und einem erhöhten Speicherverbrauch – Faktoren, die in containerisierten Umgebungen mit häufigen Neustarts besonders nachteilig sind.

4.6.2 Ahead-of-Time-Kompilierung (AOT)

Die Ahead-of-Time-Kompilierung verfolgt einen grundlegend anderen Ansatz: Der gesamte Anwendungscode wird bereits vor der Ausführung, also zum Zeitpunkt des Build-Prozesses, in nativen Maschinencode übersetzt. Quarkus nutzt hierfür GraalVM Native Image, ein Werkzeug, das Java-Anwendungen in plattformspezifische, ausführbare Binärdateien kompiliert.

Das Ergebnis ist eine native ausführbare Datei, die ohne eine laufende JVM auskommt. Dies hat folgende Vorteile:

- **Schnelle Startzeit:** Da keine JVM initialisiert werden muss und alle Optimierungen bereits zur Build-Zeit vorgenommen wurden, startet die Anwendung in Millisekunden.
- **Geringer Speicherverbrauch:** Der Wegfall der JVM-Laufzeitumgebung reduziert den RAM-Bedarf erheblich.
- **Eignung für containerisierte Umgebungen:** Kurze Startzeiten und geringer Ressourcenverbrauch machen AOT-kompilierte Anwendungen besonders geeignet für Kubernetes und ähnliche Plattformen.

Als Nachteil ist die deutlich längere Build-Zeit im Vergleich zur JIT-Kompilierung zu nennen, da die gesamte Kompilierung und Optimierung vorab stattfindet.

4.6.3 Hot Reload

Für den Entwicklungsprozess bietet Quarkus einen sogenannten Hot-Reload-Mechanismus, bekannt als *Quarkus Dev Mode*. Dabei werden Änderungen am Quellcode automatisch erkannt und ohne Neustart der Anwendung übernommen. Dies beschleunigt den Entwicklungszyklus erheblich, da zeitaufwändige Neustarts entfallen. Der Dev Mode verwendet

intern JIT-Kompilierung, um diese schnelle Rückkopplungsschleife zu ermöglichen – die AOT-Kompilierung wird lediglich für den finalen Build eingesetzt.

4.7 Quarkus in der vorliegenden Applikation

Quarkus wird in der vorliegenden Anwendung als Hauptframework für das Backend verwendet. Für die Datenbankanbindung kommt Hibernate ORM mit Panache zum Einsatz, das eine vereinfachte Abstraktion des JPA-Standards bietet und den Datenbankzugriff mit weniger Boilerplate-Code ermöglicht.

Die Anwendung wird mittels AOT-Kompilierung als native ausführbare Datei gebaut. Obwohl die Build-Zeit dadurch im Vergleich zur klassischen JIT-Kompilierung länger ausfällt, überwiegen die Vorteile im Betrieb: Die Applikation weist eine deutlich verkürzte Startzeit auf, benötigt weniger Arbeitsspeicher und ist optimal für den Einsatz in einer containerisierten Umgebung geeignet. Diese Eigenschaften sind insbesondere in Cloud-Deployments von Vorteil, wo Ressourceneffizienz und Skalierbarkeit zentrale Anforderungen darstellen.

4.8 Keycloak aus Backend-Sicht

Um die Sicherheit der Authentifizierungs- und Autorisierungsanforderungen der Anwendung zu erfüllen, wurde Keycloak als Identity- und Access-Management-Lösung integriert. Keycloak bietet eine umfassende Plattform für die Verwaltung von Benutzern, Rollen und Berechtigungen sowie die Unterstützung verschiedener Authentifizierungsprotokolle wie OpenID Connect, OAuth 2.0 und SAML.

4.9 Keycloak und JWT-Authentifizierung

Keycloak ist ein quelloffenes Identity- und Access-Management-System, das Authentifizierung und Autorisierung als eigenständigen Dienst bereitstellt. In der vorliegenden Anwendung übernimmt Keycloak die gesamte Verwaltung von Benutzeridentitäten, sodass weder im Frontend noch im Backend eine eigene Passwort- oder Benutzerverwaltung implementiert werden musste.

4.9.1 Ablauf der Authentifizierung

Der Authentifizierungsablauf folgt dem OAuth2-Protokoll mit dem Authorization Code Flow. Wenn ein Benutzer auf einen geschützten Bereich der Anwendung zugreift, leitet das Angular-Frontend ihn an Keycloak weiter. Nach erfolgreicher Anmeldung stellt Keycloak ein signiertes JSON Web Token (JWT) aus, das der Benutzer bei jeder weiteren Anfrage im `Authorization`-Header als Bearer-Token mitsendet. Dieser Ablauf wird von *Cufurovic* aus Frontend-Sicht beschrieben, während der vorliegende Abschnitt die Verarbeitung dieses Tokens im Backend behandelt.

4.9.2 JWT-Verarbeitung im Backend

Im Backend wird das vom Frontend mitgesendete JWT-Token von Quarkus mithilfe der MicroProfile JWT-Spezifikation automatisch validiert und dekodiert. Ein gültiges Token enthält unter anderem den sogenannten *Subject Claim* (`sub`), der die eindeutige Keycloak-UUID des authentifizierten Benutzers enthält. Diese UUID wird im `UserService` über die injizierte `JsonWebToken`-Instanz extrahiert:

Listing 1: Extraktion der Benutzer-UUID aus dem JWT im UserService

```
1  @Inject
2  JsonWebToken jwt;
3
4  public UUID getCurrentUserId() {
5      return UUID.fromString(jwt.getSubject());
6  }
```

Die extrahierte UUID dient als primärer Schlüssel des Benutzers in der Datenbank und stellt die Verbindung zwischen dem Keycloak-Identitätssystem und dem internen Datenbankmodell her. Da die UUID direkt aus dem signierten JWT stammt, ist eine Manipulation durch den Client ausgeschlossen.

4.9.3 Rollenbasierte Zugriffskontrolle

Neben der reinen Authentifizierung unterstützt Keycloak auch eine rollenbasierte Zugriffskontrolle (Role-Based Access Control, RBAC). Im Backend werden Endpunkte über die JAX-RS-Annotation `@RolesAllowed` auf bestimmte Rollen beschränkt. Endpunkte, die benutzerspezifische Daten liefern, sind auf die Rollen `user` und `medien-abo-users-role` beschränkt, während öffentliche Endpunkte mit `@PermitAll` für alle Aufrufer zugänglich sind. Das Frontend prüft denselben Rollenstatus, um geschützte Navigationsbereiche

ein- oder auszublenden – beide Schichten verwenden damit dieselbe Rolleninformation aus dem JWT.

5 Architektur und Systemdesign

5.1 Gesamtarchitektur der Anwendung

Die Gesamtarchitektur der Anwendung folgt einem klar getrennten Aufbau zwischen Benutzeroberfläche und serverseitiger Verarbeitung. Der Schwerpunkt meines Projektteils lag auf dem Frontend, das mit Angular als Single-Page-Application umgesetzt wurde. Das Frontend übernimmt die Darstellung der Inhalte, die Navigation zwischen den Ansichten sowie die Interaktion mit den Nutzenden. Die eigentliche Datenverarbeitung erfolgt über bereitgestellte REST-Schnittstellen, die vom Frontend angesprochen werden. Durch diese Trennung konnte die Benutzeroberfläche unabhängig von der serverseitigen Implementierung entwickelt und erweitert werden.

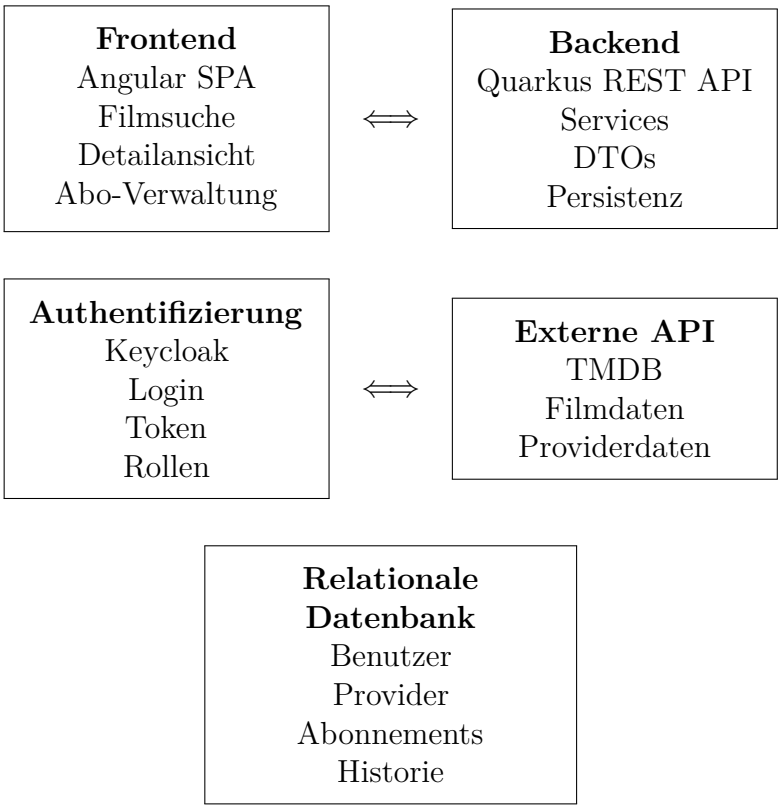

Abbildung 1: Graphische Gesamtarchitektur der Anwendung

5.2 Kommunikation zwischen Frontend und Backend

Die Kommunikation zwischen Frontend und Backend erfolgt über HTTP-basierte REST-Aufrufe. Im Frontend wurden dafür eigene Angular-Services verwendet, um Daten strukturiert von den Schnittstellen zu laden und an die Komponenten weiterzugeben [8]. Dieses Vorgehen hat den Vorteil, dass die Komponenten selbst möglichst wenig Logik für Datenzugriffe enthalten und sich stärker auf Darstellung und Interaktion konzentrieren können.

Im Projekt wurden insbesondere Services für Filmdaten, Providerdaten und die Authentifizierungsanbindung eingesetzt. Dadurch konnte die Anwendung Suchanfragen, Detailansichten und benutzerspezifische Inhalte in einer einheitlichen Struktur verarbeiten. Für das Frontend war dabei vor allem wichtig, dass externe Daten zuverlässig geladen, zwischengespeichert und in geeigneter Form in der Oberfläche dargestellt werden.

Ein typischer Ablauf innerhalb der Anwendung beginnt mit einer Benutzereingabe im Frontend, etwa einer Filmsuche. Die Suchkomponente übergibt den eingegebenen Titel an einen zuständigen Angular-Service, der daraus einen HTTP-Request an das Backend erzeugt. Das Backend verarbeitet diese Anfrage, ruft bei Bedarf externe Filmdaten über die angebundenen TMDB-Schnittstellen ab, bereitet die Antwort auf und liefert die relevanten Informationen als REST-Antwort an das Frontend zurück. Dort werden die Daten wieder vom Service entgegengenommen, in ein für die Oberfläche geeignetes Format überführt und anschließend in der Ergebnisliste dargestellt. Ein vergleichbarer Ablauf wird auch bei Detailansichten verwendet: Nach Auswahl eines Films wird anhand der Film-ID eine weitere Anfrage ausgelöst, damit zusätzliche Informationen wie Beschreibung, Laufzeit, Genres und verfügbare Anbieter geladen werden können.

Die Trennung zwischen Komponente und Service war dabei für die Wartbarkeit besonders wichtig. Komponenten steuern Benutzerinteraktionen wie Suche, Navigation oder Auswahl eines Films, während Services die eigentliche Kommunikation mit dem Backend bündeln. Dadurch konnten Suchlogik, Detailaufrufe und Providerabfragen an zentraler Stelle verwaltet werden, ohne dieselben HTTP-Abläufe in mehreren Komponenten zu wiederholen. Gleichzeitig ließ sich die Oberfläche dadurch leichter erweitern, weil Datenquellen und Darstellung voneinander entkoppelt blieben.

Bei benutzerspezifischen Daten reicht ein normaler HTTP-Aufruf jedoch nicht aus. Solche Daten können nur dann vom Backend geladen werden, wenn das Frontend ein

gültiges Bearer-Token mitsendet. Dieses Token wird nach erfolgreicher Anmeldung über Keycloak bereitgestellt und bei geschützten Anfragen im Authorization-Header übertragen. Auf diese Weise kann das Backend prüfen, ob der Benutzer authentifiziert ist und Zugriff auf die angeforderten Daten besitzt [12, 13].

Das betrifft insbesondere Aufrufe zur persönlichen Abo-Verwaltung. Wenn gespeicherte Abonnements geladen, geändert oder neu gespeichert werden, sendet das Frontend die Anfrage nicht nur an den passenden Backend-Endpunkt, sondern überträgt zusätzlich das Bearer-Token im `Authorization`-Header. Erst dadurch kann das Backend die Anfrage dem aktuell angemeldeten Benutzer zuordnen und ausschließlich dessen Daten verarbeiten. Aus Frontend-Sicht war deshalb nicht nur der reine Datenabruf relevant, sondern das verlässliche Zusammenspiel aus Service-Aufruf, Token-Übergabe und anschließender Aktualisierung der Oberfläche.

5.3 API-Integration externer Movie-Datenbanken

Ein wichtiger Bestandteil der Anwendung ist die Darstellung von Filminformationen und Streaming-Anbietern. Aus Sicht des Frontends bestand die Aufgabe darin, diese Informationen benutzerfreundlich aufzubereiten und in unterschiedlichen Ansichten darzustellen. Dazu gehören insbesondere die Filmübersicht, die Detailansicht einzelner Filme sowie die Anzeige verfügbarer Anbieter.

Die Filmdaten werden über bereitgestellte Schnittstellen geladen und in Angular-Komponenten eingebunden. Für die Benutzeroberfläche war dabei entscheidend, dass Suchergebnisse übersichtlich dargestellt, Detailinformationen verständlich aufbereitet und Navigationswechsel zwischen Listen- und Detailansicht flüssig umgesetzt werden. Da die Anwendung als Single-Page-Application realisiert wurde, konnte dieser Wechsel ohne vollständigen Seitenneuaufbau erfolgen [9]. Als externe Grundlage diente dabei die Dokumentation der TMDB API, insbesondere für Authentifizierung, Filmdaten und Anbieterinformationen [2, 3].

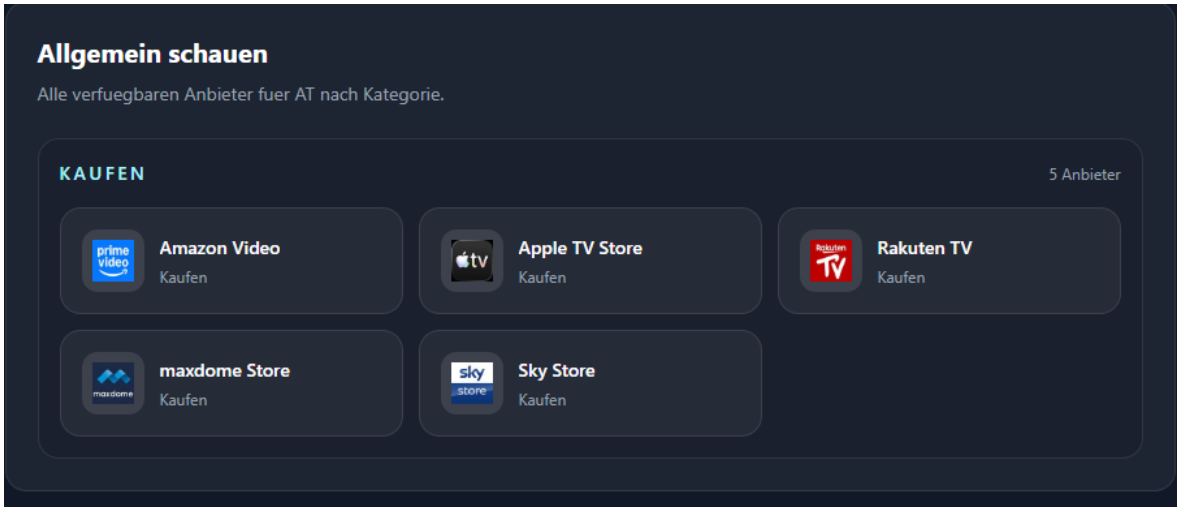

Abbildung 2: Anbieteranzeige innerhalb der Film-Detailansicht

5.4 Sicherheits- und Datenschutzkonzept

Aus Frontend-Sicht spielt das Sicherheits- und Datenschutzkonzept vor allem bei der Zugriffskontrolle auf geschützte Bereiche eine Rolle. Öffentliche Inhalte der Anwendung können ohne Anmeldung aufgerufen werden, während persönliche Funktionen wie die Verwaltung eigener Streaming-Abonnements nur nach erfolgreicher Authentifizierung zugänglich sind. Für diesen Zweck wurde Keycloak in das Frontend integriert [12, 13].

Ein wichtiger Bestandteil dieses Konzepts ist, dass personenbezogene oder benutzerspezifische Daten nicht öffentlich abrufbar sind. Das Backend stellt diese Informationen nur dann bereit, wenn bei der Anfrage ein gültiges Bearer-Token übertragen wird. Das Frontend übernimmt dabei die Aufgabe, den Authentifizierungsstatus zu verwalten und das Token bei geschützten Requests mitzuschicken. Dadurch wird sichergestellt, dass persönliche Inhalte nur autorisierten Benutzern angezeigt werden [12].

Die Angular-Anwendung erkennt den Authentifizierungsstatus der Nutzenden und steuert darauf basierend den Zugriff auf geschützte Routen. Dadurch wird sichergestellt, dass sensible benutzerspezifische Inhalte nicht ohne Anmeldung angezeigt werden. Gleichzeitig verbessert diese Struktur die Benutzerfreundlichkeit, da sich öffentliche und persönliche Bereiche klar voneinander trennen lassen. Die eigentliche Verwaltung von Zugangsdaten erfolgt dabei nicht im Frontend selbst, sondern über das externe Authentifizierungssystem.

5.5 Datenbanken

Um Dateneingaben persistent zu speichern und strukturiert zu verarbeiten, ist der Einsatz einer Datenbank unerlässlich. Eine Datenbank stellt die zentrale Instanz für die Speicherung, Organisation und Verwaltung von Daten dar. Sie ermöglicht nicht nur das einfache Ablegen von Informationen, sondern auch deren gezielte Abfrage, Aktualisierung und Löschung. Um auf die gespeicherten Daten zugreifen und diese verwalten zu können, wird ein sogenanntes Datenbankmanagementsystem (DBMS) eingesetzt, das als Vermittler zwischen der Anwendung und den physisch gespeicherten Daten fungiert.

5.5.1 Datenbank definition

Eine Datenbank ist eine strukturierte Sammlung von Daten, die so organisiert ist, dass sie effizient gespeichert, abgerufen und verwaltet werden kann. Im Gegensatz zur einfachen Dateiablage bietet eine Datenbank Mechanismen zur Vermeidung von Redundanzen, zur Sicherstellung der Datenkonsistenz sowie zur gleichzeitigen Nutzung durch mehrere Benutzer oder Anwendungen.

Grundlegende Begriffe im Datenbankumfeld sind:

- **Entität**: Ein eindeutig identifizierbares Objekt der realen Welt, das in der Datenbank abgebildet wird (z. B. ein Benutzer oder ein Streaming-Anbieter).
- **Attribut**: Eine Eigenschaft einer Entität (z. B. Name oder ID).
- **Relation**: Eine Tabelle in einer relationalen Datenbank, die Entitäten eines bestimmten Typs speichert.
- **Primärschlüssel**: Ein Attribut oder eine Kombination von Attributen, das eine Entität eindeutig identifiziert.
- **Fremdschlüssel**: Ein Attribut, das auf den Primärschlüssel einer anderen Tabelle verweist und damit Beziehungen zwischen Entitäten abbildet.

5.5.2 Datenbank Managment System - DBMS

Ein Datenbankmanagementsystem (DBMS) ist eine Software, die den Zugriff auf eine Datenbank verwaltet und kontrolliert. Es stellt die Schnittstelle zwischen der

Anwendung und den physisch gespeicherten Daten dar und übernimmt Aufgaben wie die Speicherverwaltung, die Zugriffskontrolle, die Transaktionsverwaltung sowie die Sicherstellung der Datenkonsistenz.

Ein DBMS besteht im Wesentlichen aus zwei zentralen Komponenten. Der *physische Datenbestand* umfasst die tatsächlich auf einem Datenträger gespeicherten Daten. Der *Systemkatalog* – auch Data Dictionary genannt – enthält Metainformationen über die Struktur der Datenbank, also Informationen über Tabellen, Spalten, Datentypen, Beziehungen und Zugriffsrechte. Mithilfe des Systemkatalogs kann das DBMS Anfragen korrekt interpretieren und die gespeicherten Daten strukturiert zurückgeben.

Zu den bekanntesten DBMS zählen PostgreSQL, MySQL, Oracle Database und Microsoft SQL Server im relationalen Bereich sowie MongoDB und Redis im nicht-relationalen Bereich.

5.6 Datenbankkonzept und architektonische Entscheidungen

Für das Backend der Anwendung spielt die Wahl der geeigneten Datenbanktechnologie eine zentrale Rolle. Trotz vorhandener Kenntnisse im Bereich nicht-relationaler Datenbanken (NoSQL) – wie etwa MongoDB oder Vektordatenbanken – wurde im Rahmen dieses Projekts bewusst eine relationale Datenbank eingesetzt.

Die Entscheidung für ein relationales Datenbanksystem basiert auf mehreren Faktoren. Die Anwendung weist eine klar strukturierte und logisch getrennte Datenarchitektur auf, wodurch sich die Daten und deren Beziehungen gut in einem relationalen Modell abbilden lassen. Entitäten und ihre Relationen konnten somit eindeutig definiert und konsistent strukturiert werden. Dieses Vorgehen entspricht dem etablierten Standard bei Anwendungen mit stark strukturierten Daten und klar definierten Geschäftsprozessen.

Ein weiterer wesentlicher Aspekt ist die Skalierbarkeit des Projekts. Relationale Datenbanken bieten durch Normalisierung, Transaktionssicherheit (ACID-Prinzip) und klar definierte Beziehungen eine stabile Grundlage für zukünftige Erweiterungen. Voraussetzung für eine nachhaltige Skalierbarkeit ist jedoch eine sorgfältig durchdachte Datenbankstruktur. Eine frühzeitig geplante und konsistente Modellierung – beispiels-

weise durch ein Entity-Relationship-Diagramm (ERD) – ist essenziell, um spätere strukturelle Anpassungen zu minimieren.

5.6.1 Herausforderungen im Entwicklungsprozess

Im Verlauf der Diplomarbeit traten mehrere strukturelle Herausforderungen auf, die im Sinne einer transparenten Dokumentation offen dargestellt werden sollen.

Zu Beginn der Implementierungsphase wurde ohne vollständig ausgearbeitetes Gesamtkonzept mit der Programmierung begonnen. Das Entity-Relationship-Diagramm wurde parallel zur Entwicklung erstellt, wodurch sich die Datenbankstruktur dynamisch entwickelte. Dieses iterative Vorgehen führte zwar zu schneller praktischer Umsetzung, hatte jedoch den Nachteil, dass strukturelle Änderungen im Nachhinein größeren Anpassungsaufwand verursachten.

```
+---------------------+
|        User         |
+---------------------+
| id (PK)             |
| username            |
| email               |
| password            |
| providers           |  ← String Set (e.g. "Netflix,Disney+,Prime")
| createdAt           |
+---------------------+
            |
            | 1
            |
            |------< N
                     |
             +----------------------+
             |        Movie         |
             +----------------------+
             | id (PK)              |
             | title                |
             | genre                |
             | releaseYear          |
             | providerName         | ← Stored as String (duplicate data)
             | userId (FK → User.id)|
             +----------------------+
```

Abbildung 3: Erstes Entity-Relationship-Diagramm der Anwendung

Eine weitere wesentliche Herausforderung ergab sich durch die späte Integration von Keycloak als Identity- und Access-Management-Lösung. Ursprünglich war die Benutzerverwaltung eigenständig implementiert worden. Durch die nachträgliche Einführung

von Keycloak mussten die bestehenden User-Strukturen grundlegend überarbeitet werden. Dies hatte zur Folge, dass die Benutzerentitäten angepasst werden mussten, die Authentifizierungs- und Autorisierungslogik neu strukturiert wurde sowie abhängige Klassen und Beziehungen refaktoriert und teilweise neu implementiert werden mussten. Diese Umstellung führte zu zusätzlichem Entwicklungsaufwand, zeigte jedoch zugleich die Bedeutung einer frühzeitigen Architekturentscheidung – insbesondere im Bereich der Sicherheits- und Authentifizierungsmechanismen.

Abbildung 4: Überarbeitetes Entity-Relationship-Diagramm nach Integration von Keycloak

5.6.2 Modellierung der Many-to-Many-Beziehung und strukturelle Weiterentwicklung

Im weiteren Verlauf der Systemarchitektur zeigte sich deutlich der Mehrwert einer sauberen relationalen Modellierung, insbesondere bei der Abbildung von Many-to-Many-Beziehungen.

Im Kontext der vorliegenden Applikation – einem Mediatheken-Abonnement-Manager – besteht zwischen den Entitäten *User* und *Provider* keine einfache 1:n-Beziehung, sondern eine n:m-Beziehung. Ein Benutzer kann mehrere Anbieter abonnieren, während ein Anbieter von mehreren Benutzern genutzt werden kann.

Die Einführung einer eigenständigen Relationstabelle (`UserProviderSubscription`) dient hierbei nicht ausschließlich der technischen Verknüpfung zweier Fremdschlüssel. Vielmehr übernimmt diese Tabelle eine fachlich zentrale Rolle innerhalb des Domänenmodells. In der Relationstabelle werden zusätzliche Attribute gespeichert, die die konkrete Ausprägung der Beziehung beschreiben:

- Beginn des Abonnements (Startdatum)
- Ende des Abonnements (Enddatum)
- Zahlungszyklus (Enum: monatlich oder jährlich)

Diese Attribute gehören semantisch weder ausschließlich zum Benutzer noch ausschließlich zum Anbieter, sondern definieren die konkrete Vertragsbeziehung zwischen beiden Entitäten. Die Modellierung über eine separate Relationstabelle ermöglicht somit eine fachlich korrekte Abbildung der Geschäftslogik, die Erweiterbarkeit um weitere beziehungsbezogene Attribute – etwa Preis, Rabatt oder Kündigungsstatus – sowie eine klare Trennung der Verantwortlichkeiten innerhalb des Datenmodells. Gerade an dieser Stelle wird deutlich, dass relationale Datenbanken ihre Stärke bei strukturierten und stark beziehungsorientierten Datenmodellen ausspielen.

5.6.3 Reflexion der anfänglichen Modellierungsentscheidung

Rückblickend stellt sich die anfängliche Entscheidung, Anbieter als `Set<String>` direkt innerhalb der User-Entität zu speichern, als struktureller Fehler heraus. Diese vereinfachte Modellierung war zwar für einen ersten Prototyp funktional ausreichend, führte jedoch zu mehreren konzeptionellen Einschränkungen:

1. Es konnten keine anbieterspezifischen Metadaten gespeichert werden (z. B. Website, Kategorie oder Beschreibung).
2. Die konkrete Ausgestaltung der Beziehung – Startdatum, Zahlungszyklus etc. – war nicht modellierbar.
3. Eine konsistente Datenhaltung war nicht gewährleistet, da String-Werte fehleranfällig und nicht referenziell abgesichert sind.
4. Erweiterungen des Systems hätten tiefgreifende strukturelle Änderungen erfordert.

Insbesondere der Umstand, dass beziehungsrelevante Informationen nicht abgebildet werden konnten, verdeutlicht die Problematik dieser frühen Designentscheidung. Die Speicherung als String-Set widersprach den Grundprinzipien der Normalisierung und verhinderte eine skalierbare Weiterentwicklung des Systems. Erst durch die Einführung einer eigenständigen `Provider`-Entität sowie einer Relationstabelle konnte das Datenmodell den fachlichen Anforderungen der Applikation gerecht werden. Diese Umstrukturierung stellt einen wesentlichen architektonischen Reifeprozess im Projektverlauf dar.

5.7 Backend-Architektur

Die Backend-Architektur der vorliegenden Anwendung folgt einem mehrschichtigen Aufbau, der eine klare Trennung der Verantwortlichkeiten gewährleistet. Dieses Architekturmuster wird auch als *Layered Architecture* bezeichnet. Jede Schicht übernimmt eine klar definierte Aufgabe, wodurch die Wartbarkeit, Testbarkeit und Erweiterbarkeit der Anwendung verbessert wird. Eine eingehende HTTP-Anfrage durchläuft dabei die Resource-Schicht, wird über DTOs weitergeleitet, von der Service-Schicht verarbeitet und schließlich über das Repository auf die Datenbankentitäten abgebildet.

5.7.1 Entity

Die Entity-Schicht repräsentiert die Datenmodelle einer Anwendung. Jede Entity-Klasse bildet die Struktur einer Datenbanktabelle ab und wird mithilfe von Annotationen der Jakarta Persistence API (JPA) mit der Datenbank verknüpft. Entity-Klassen enthalten ausschließlich die Datenbankstruktur sowie einfache Zugriffsmethoden. Geschäftslogik

hat in dieser Schicht keine Berechtigung, da dies die Trennung der Verantwortlichkeiten verletzen würde.

5.7.2 Repository

Die Repository-Schicht kapselt den Zugriff auf die Datenbank und stellt eine Abstraktion zwischen der Datenzugriffslogik und den darüberliegenden Schichten dar. Sie definiert die grundlegenden CRUD-Operationen (Create, Read, Update, Delete) sowie komplexere Datenbankabfragen. Durch diese Trennung bleibt die Geschäftslogik unabhängig von der konkreten Datenbankimplementierung, was einen späteren Austausch der Persistenzschicht erleichtert und die Testbarkeit der Anwendung erhöht.

5.7.3 Service

Die Service-Schicht beinhaltet die Geschäftslogik der Anwendung und fungiert als Vermittler zwischen der Resource- und der Repository-Schicht. Sie ist verantwortlich für die Verarbeitung und Transformation von Daten, die Koordination mehrerer Repository-Zugriffe sowie die Kommunikation mit externen Systemen. Die Service-Schicht stellt sicher, dass die Geschäftsregeln der Anwendung zentral und unabhängig von der Präsentations- oder Datenzugriffsschicht definiert sind.

5.7.4 Resource

Die Resource-Schicht ist für die Bereitstellung der RESTful-Endpunkte verantwortlich. Sie nimmt eingehende HTTP-Anfragen entgegen, delegiert die Verarbeitung an die zuständigen Service-Klassen und gibt die entsprechenden Antworten an den Client zurück. Die Resource-Schicht enthält dabei selbst keine Geschäftslogik, sondern fungiert ausschließlich als Schnittstelle zwischen dem Backend und den konsumierenden Clients. Durch diese strikte Trennung bleibt die Schicht schlank und leicht austauschbar.

5.7.5 DTO – Data Transfer Objects

Data Transfer Objects (DTOs) sind speziell zugeschnittene Datenobjekte, die dem gezielten Datenaustausch zwischen verschiedenen Schichten oder zwischen Backend und Client dienen. Anstatt vollständige Entity-Objekte direkt weiterzugeben, werden DTOs

verwendet, die ausschließlich jene Felder enthalten, die für den jeweiligen Anwendungsfall erforderlich sind.

DTOs erfüllen dabei zwei wesentliche Zwecke. Erstens erhöhen sie die Sicherheit der Anwendung: Sensible Felder einer Entity, wie etwa Passwort-Hashes oder interne Systemfelder, werden nicht versehentlich nach außen weitergegeben. Zweitens wird eine Entkopplung zwischen der internen Datenstruktur und der nach außen sichtbaren API-Struktur erreicht. Änderungen am Datenbankmodell haben dadurch keinen direkten Einfluss auf die API-Schnittstelle, was die Stabilität der Anwendung langfristig sichert.

5.8 Implementierung des Backends in der vorliegenden Applikation

Die im vorherigen Abschnitt beschriebenen Architekturprinzipien wurden in der vorliegenden Anwendung konsequent umgesetzt. Das Backend basiert auf Quarkus und verwendet Hibernate ORM für die Datenbankanbindung. Die folgende Beschreibung erläutert, wie die einzelnen Schichten konkret implementiert wurden.

5.8.1 Entity

Die Datenbankstruktur der Anwendung wird durch fünf Entity-Klassen sowie ein Enum abgebildet, die gemeinsam das Domänenmodell der Anwendung repräsentieren.

Die zentrale Entität ist `UserMovieDB`, welche einen registrierten Benutzer der Anwendung darstellt. Als Primärschlüssel wird dabei keine automatisch generierte Datenbank-ID verwendet, sondern die `UUID` des Benutzers aus dem Identity-Provider Keycloak. Dies stellt sicher, dass die interne Benutzerrepräsentation direkt mit dem Authentifizierungssystem verknüpft ist.

Die Entität `Provider` repräsentiert einen Streaming-Anbieter wie etwa Netflix oder Amazon Prime. Neben einer internen Datenbank-ID speichert sie die `tmdbProviderId`, also die eindeutige ID des Anbieters in der TMDB API. Diese dient als Brücke zwischen den intern gespeicherten Providern und den von der externen API gelieferten Daten. Jedem Provider kann zusätzlich eine Liste von `CountryPriority`-Einträgen zugeordnet werden, die die Anzeigepriorität des Anbieters je nach Land abbilden.

Die Verknüpfung zwischen Benutzer und Streaming-Anbieter erfolgt über die Entität `UserProviderSubscription`. Diese speichert neben den Fremdschlüsseln auf `UserMovieDB` und `Provider` auch den Abrechnungszyklus des Abonnements sowie das letzte Abrechnungsdatum. Der Abrechnungszyklus wird über das Enum `BillingCycle` mit den Ausprägungen `MONTHLY` und `YEARLY` abgebildet. Eine eindeutige Kombination aus Benutzer und Anbieter wird durch eine `UniqueConstraint`-Annotation auf Datenbankebene sichergestellt, sodass ein Benutzer denselben Anbieter nicht mehrfach abonnieren kann. Die Entität stellt zudem eine berechnete Methode `getNextBillingDate()` bereit, die basierend auf dem Abrechnungszyklus das nächste Fälligkeitsdatum ermittelt.

Die Entität `History` speichert den Filmschauerverlauf eines Benutzers. Sie referenziert den jeweiligen Benutzer über einen Fremdschlüssel sowie die `tmdbMovieId` des gesehenen Films. Der Zeitpunkt des Ansehens wird automatisch beim Erstellen eines `History`-Eintrags auf den aktuellen Zeitstempel gesetzt.

5.8.2 Repository

Die Repository-Schicht der Anwendung besteht aus drei Klassen, die jeweils den Datenbankzugriff für eine bestimmte Entität kapseln.

Das `UserMovieDBRepository` implementiert das `PanacheRepository`- Interface und stellt spezialisierte Abfragemethoden bereit. Die Methode `findByUUID` sucht einen Benutzer anhand seiner Keycloak-UUID und gibt ein `Optional` zurück, um den Fall eines nicht vorhandenen Benutzers typsicher zu behandeln. Die Methode `getUserProviderTmdbIds` liefert alle TMDB-Provider-IDs der Abonnements eines bestimmten Benutzers als `Set`, das anschließend im Service-Layer für den Abgleich mit der TMDB API verwendet wird. Besonders hervorzuheben ist die Methode `getProvidersWithOwnership`, die mithilfe einer JPQL-Abfrage alle Provider aus der Datenbank lädt und dabei über einen `LEFT JOIN` mit den Abonnements des Benutzers verknüpft. Das Ergebnis enthält für jeden Provider ein Flag, das angibt, ob der Benutzer diesen abonniert hat. Dadurch kann das Frontend mit einem einzigen Request alle relevanten Informationen abrufen. Darüber hinaus enthält das Repository die Methode `updateSubscriptions`, die die Abonnements eines Benutzers anhand einer übergebenen Liste von `SubscriptionUpdateDTO`-Objekten aktualisiert. Dabei werden neue Abonnements angelegt, bestehende aktualisiert und nicht mehr enthaltene entfernt.

Das `ProviderRepository` verwaltet die Streaming-Anbieter in der Datenbank. Die zentrale Methode ist `upsertFromTmdb`, die beim Anwendungsstart aufgerufen wird: Sie prüft, ob ein Provider mit der gegebenen `tmdbProviderId` bereits existiert, und aktualisiert dessen Daten oder legt einen neuen Eintrag an. Dadurch wird eine konsistente Synchronisierung der Provider-Daten mit der TMDB API gewährleistet.

Das `HistoryRepository` implementiert ebenfalls das `PanacheRepository`-Interface und stellt Methoden für den Zugriff auf den Filmschauerverlauf bereit. Die Methode `findByUserId` gibt alle History-Einträge eines Benutzers sortiert nach dem Zeitpunkt des Ansehens zurück. Über `findByUserIdLimit` kann die Anzahl der zurückgegebenen Einträge begrenzt werden, was eine einfache Paginierung ermöglicht. Die Methode `hasWatchedMovie` prüft, ob ein Benutzer einen bestimmten Film bereits gesehen hat.

5.8.3 Service

Die Service-Schicht der Anwendung besteht aus drei Klassen, die jeweils einen klar abgegrenzten Verantwortungsbereich abdecken.

Der `UserService` ist für die Verwaltung des aktuell authentifizierten Benutzers zuständig. Er liest die Keycloak-UUID aus dem JWT-Token aus und prüft über das `UserMovieDBRepository`, ob der Benutzer bereits in der Datenbank existiert. Ist dies nicht der Fall, wird automatisch ein neuer `UserMovieDB`-Eintrag angelegt. Dieses Muster wird als *Lazy User Creation* bezeichnet und stellt sicher, dass beim ersten authentifizierten Zugriff eines neuen Benutzers automatisch ein Datenbankprofil erstellt wird, ohne dass eine explizite Registrierung erforderlich ist.

Der `TmdbService` kapselt die gesamte Kommunikation mit der externen TMDB API. Die HTTP-Anfragen werden über die Bibliothek OkHttp durchgeführt, die empfangenen JSON-Antworten mit Jackson geparst und auf DTOs abgebildet. Der API-Schlüssel wird nicht im Quellcode hinterlegt, sondern über den Quarkus-Konfigurationsmechanismus mittels `@ConfigProperty` injiziert, was die Sicherheit der Anwendung erhöht. Der Service stellt Methoden zum Abrufen von Filminformationen per ID oder Suchbegriff, zum Laden verfügbarer Streaming-Anbieter sowie zum Abrufen trendender Filme bereit. Die Methode `getFilteredProvidersForUser` kombiniert dabei externe API-Daten mit internen Datenbankdaten: Sie ruft die verfügbaren Provider eines Films von der TMDB API ab, gleicht diese mit den vom Benutzer abonnierten Anbietern aus der

Datenbank ab und setzt das Flag `ownedByUser` im zurückgegebenen `ProviderInfoDTO`. Die Ergebnisliste wird abschließend so sortiert, dass die vom Benutzer abonnierten Anbieter an erster Stelle erscheinen.

Der `ProviderSyncService` wird beim Start der Anwendung automatisch ausgeführt, indem er auf das Quarkus-Lifecycle-Event `StartupEvent` reagiert. Er ruft über den `TmdbService` alle verfügbaren Streaming-Anbieter von der TMDB API ab und synchronisiert diese über die `upsertFromTmdb`-Methode des `ProviderRepository` in die lokale Datenbank. Dadurch ist sichergestellt, dass die Anwendung stets eine aktuelle Liste aller Provider vorhält, ohne dass ein manueller Synchronisierungsschritt erforderlich ist. Der gesamte Vorgang wird in einer Transaktion ausgeführt, was durch die `@Transactional`-Annotation sichergestellt wird.

5.8.4 Konfiguration über application.properties

Die zentrale Konfiguration des Backends erfolgt über die Datei `src/main/resources/application.properties`. Dort werden unter anderem der TMDB-API-Schlüssel, die Datenbankverbindung über `quarkus.datasource.*` sowie die OIDC- und Keycloak-Parameter über `quarkus.oidc.*` definiert. Zusätzlich werden an dieser Stelle öffentliche Routen ohne Authentifizierung festgelegt sowie weitere anwendungsrelevante Einstellungen wie CORS, SQL-Logging und die Hibernate-Konfiguration verwaltet. Die projektbezogene Bündelung dieser Angaben erleichtert die Wartung und sorgt dafür, dass Infrastruktur-, Sicherheits- und Laufzeitparameter zentral nachvollziehbar bleiben.

5.8.5 Resource

Die Resource-Schicht der Anwendung besteht aus drei produktiven Resource-Klassen, die die REST-Endpunkte des Backends definieren.

Die `MovieResource` stellt Endpunkte unter dem Basispfad `/api/movies` bereit. Sie ermöglicht das Abrufen von Filmdaten anhand einer ID oder eines Suchbegriffs, das Laden verfügbarer Streaming-Anbieter für einen Film sowie das Abrufen trendender Filme. Einige Endpunkte sind öffentlich zugänglich und mit `@PermitAll` annotiert, während der Endpunkt `/api/movies/{id}/providers/for-user` eine gültige Authentifizierung erfordert und über `@RolesAllowed` auf die Rollen `user` und `medien-abo-users-role`

beschränkt ist. Bei diesem Endpunkt wird die Benutzer-ID direkt aus dem JWT-Token extrahiert, sodass keine User-ID in der URL übermittelt werden muss.

Die `UserResource` stellt Endpunkte unter dem Basispfad `/api/users` bereit und ist vollständig authentifizierungsgeschützt. Sie bietet Funktionalitäten zur Verwaltung des eigenen Benutzerprofils, der Streaming-Abonnements sowie der Filmschauhistorie. Über `GET /api/users/me` kann der aktuelle Benutzer seine Profildaten abrufen, wobei bei einem erstmaligen Zugriff automatisch ein Datenbankprofil angelegt wird. Der Endpunkt `PUT /api/users/me/subscriptions` ermöglicht das vollständige Aktualisieren der Abonnements in einem einzigen Request. Die History-Endpunkte erlauben das Hinzufügen, Abrufen und Löschen von Einträgen im Filmschauerverlauf. Beim Löschen wird explizit geprüft, ob der Eintrag dem authentifizierten Benutzer gehört, bevor die Operation ausgeführt wird.

Die `ProviderRessource` stellt Endpunkte unter `/api/providers` bereit. Sie ermöglicht das Abrufen aller in der Datenbank gespeicherten Streaming-Anbieter sowie das manuelle Auslösen einer Synchronisierung mit der TMDB API über den Endpunkt `GET /api/providers/sync`.

Tabelle 1 gibt einen kompakten Überblick über die zentralen REST-Endpunkte des Backends, deren Zugriffsschutz und ihre jeweilige Aufgabe.

5.8.6 DTO – Data Transfer Objects

In der vorliegenden Anwendung werden DTOs konsequent eingesetzt, um die Kommunikation zwischen den Schichten zu strukturieren und interne Datenbankstrukturen zu kapseln.

Der `ProviderInfoDTO` wird bei der Abfrage von Streaming-Anbietern eines Films verwendet. Er enthält die TMDB-Provider-ID, den Anbieternamen, die URL des Provider-Logos sowie das Flag `ownedByUser`, das angibt, ob der jeweilige Benutzer diesen Anbieter abonniert hat. Die vollständige `Provider`-Entity sowie interne Datenbankfelder werden dabei nicht weitergegeben. Dieses Flag wird im Backend gezielt gesetzt, damit das Frontend abonnierte Anbieter in der Film-Detailansicht hervorheben und in der Darstellung priorisiert anordnen kann.

Der `ProviderWithOwnership`-DTO wird vom `UserMovieDBRepository` direkt über eine JPQL-Konstruktorabfrage befüllt und an den Client übermittelt. Er enthält neben

Methode	Endpunkt	Zugriff	Beschreibung
GET	`/api/movies/id/{id}`	öffentlich	Abruf von Filmdaten anhand einer TMDB-ID
GET	`/api/movies/name/{name}`	öffentlich	Filmsuche über den Titel
GET	`/api/movies/trending/{window}`	öffentlich	Abruf aktueller Trending-Filme
GET	`/api/movies/{id}/providers/filtered`	öffentlich	Länderbezogene Provider eines Films
GET	`/api/movies/{id}/providers/for-user`	geschützt	Providerliste mit Benutzerbezug für den aktuell angemeldeten Benutzer
GET	`/api/users/me`	geschützt	Abruf des eigenen Benutzerprofils
GET	`/api/users/me/providers`	geschützt	Abruf aller Provider mit Ownership-Informationen des Benutzers
PUT	`/api/users/me/subscriptions`	geschützt	Vollständiges Aktualisieren der gespeicherten Abonnements
POST	`/api/users/me/history`	geschützt	Hinzufügen eines Films zur persönlichen Historie
GET	`/api/users/me/history`	geschützt	Abruf der persönlichen Historie
DELETE	`/api/users/me/history/{id}`	geschützt	Löschen eines Eintrags aus der persönlichen Historie
GET	`/api/providers`	öffentlich	Abruf aller in der Datenbank gespeicherten Provider

Tabelle 1: Zentrale REST-Endpunkte des Backends mit Zugriffsschutz

den Provider-Stammdaten auch den Abrechnungszyklus und das letzte Abrechnungsdatum des Abonnements, sofern der Benutzer den jeweiligen Anbieter abonniert hat.

Der `SubscriptionUpdateDTO` dient als Eingabe-DTO für den `PUT /api/users/me/subscriptions` Endpunkt. Er enthält die TMDB-Provider-ID, den gewünschten Abrechnungszyklus sowie das Abrechnungsdatum und ermöglicht so die vollständige Aktualisierung eines Abonnements in einem einzigen Objekt.

Der `TrendingMovieDTO` bündelt alle relevanten Informationen eines trendenden Films – Titel, Poster-URL, Kurzbeschreibung, Erscheinungsdatum, Popularitätswert sowie Bewertungsdurchschnitt und Bewertungsanzahl – in einem kompakten Objekt, das direkt an den Client übermittelt wird.

Der `HistoryDTO` dient der strukturierten Rückgabe von History-Einträgen an den Client. Er wird über eine statische `from(History)`-Methode aus der gleichnamigen Entity erstellt und stellt sicher, dass keine internen JPA-Proxies oder lazy-geladenen Assoziationen versehentlich serialisiert werden.

Der `ProviderApi`-Record dient als internes Transferobjekt zwischen dem `TmdbService` und dem `ProviderSyncService`. Er bildet die Struktur der TMDB-API-Antwort ab und ermöglicht eine typsichere Weiterverarbeitung der Rohdaten, bevor diese in die Datenbank geschrieben werden.

5.9 Externe APIs und die Entscheidung für die TMDB API

Ein zentraler Bestandteil der vorliegenden Anwendung ist die Darstellung von Filminformationen und Streaming-Anbietern. Für die Bereitstellung dieser Daten wurde eine externe API eingesetzt. Der folgende Abschnitt erläutert, was eine API ist, warum eine externe Lösung gegenüber einer eigenen Datenbankpflege bevorzugt wurde und nach welchen Kriterien die Entscheidung für die TMDB API getroffen wurde.

5.9.1 Definition und Funktion einer API

Der Begriff API steht für *Application Programming Interface* und bezeichnet eine Schnittstelle, über die eine Anwendung auf Daten oder Funktionalitäten eines externen Systems zugreifen kann. Eine API definiert dabei, welche Anfragen gestellt werden können, in welchem Format die Antworten zurückgegeben werden und wie die Authentifizierung gegenüber dem System erfolgt.

Im Kontext von Webanwendungen werden APIs häufig als REST-Schnittstellen realisiert, die über HTTP-Anfragen angesprochen werden und ihre Antworten im JSON-Format zurückgeben. Dieses Prinzip findet sich auch in der vorliegenden Anwendung selbst wieder: Das Quarkus-Backend stellt dem Angular-Frontend ebenfalls eine REST-API bereit und agiert damit gegenüber dem Frontend in derselben Rolle wie eine externe API gegenüber dem Backend. Der Unterschied liegt lediglich darin, dass das eigene Backend vollständig unter der Kontrolle des Entwicklungsteams liegt, während eine externe API von einem Drittanbieter betrieben und gepflegt wird.

5.9.2 Eigene Datenhaltung vs. Auslagerung an eine externe API

Bei der Konzeption der Anwendung stellte sich grundlegend die Frage, ob Filmdaten eigenständig in der Datenbank verwaltet oder über eine externe API bezogen werden sollen. Beide Ansätze haben spezifische Vor- und Nachteile, die im Folgenden gegenübergestellt werden.

Eigene Datenhaltung

Bei einer vollständig eigenständigen Datenhaltung würden alle Filmdaten – Titel, Beschreibungen, Erscheinungsdaten, Bewertungen, Poster und Streaming-Verfügbarkeiten – in der eigenen Datenbank gespeichert und gepflegt werden. Dies hätte den Vorteil, dass keinerlei Abhängigkeit von einem externen Anbieter besteht, keine API-Schlüssel mit limitierten Aufrufkontingenten benötigt werden und alle Daten vollständig kontrolliert werden können.

Dem stehen jedoch erhebliche Nachteile gegenüber. Der globale Filmbestand umfasst hunderttausende Einträge, was zu einer enormen Datenmenge führt. Hinzu kommt der kontinuierliche Wartungsaufwand: Neue Filme müssten manuell oder über eigene Crawler erfasst, Streaming-Verfügbarkeiten laufend aktualisiert und veraltete Daten bereinigt werden. Dieser Aufwand übersteigt den Rahmen eines Diplomarbeitsprojekts bei weitem und würde den Fokus von der eigentlichen Applikationsentwicklung ablenken.

Auslagerung an eine externe API

Die Auslagerung der Filmdaten an eine spezialisierte externe API vermeidet die genannten Nachteile. Der Datenbankbetrieb, die Pflege des Filmbestands sowie die Aktualisierung von Streaming-Verfügbarkeiten werden vollständig vom API-Anbieter übernommen. Die eigene Anwendung konsumiert lediglich die bereitgestellten Daten und kann sich auf ihre Kernfunktionalität konzentrieren.

Als Nachteil ist die Abhängigkeit vom externen Anbieter zu nennen: Bei Änderungen an der API-Struktur oder bei einer Einstellung des Dienstes wäre eine Anpassung der Anwendung erforderlich. Zudem sind die Aufrufkontingente je nach Tarifmodell begrenzt. Für den Anwendungsfall des vorliegenden Projekts überwiegen jedoch die Vorteile der Auslagerung deutlich, weshalb diese Lösung gewählt wurde.

Kriterium	Eigene Datenhaltung	Externe API
Datenkontrolle	Vollständig	Abhängig vom Anbieter
Wartungsaufwand	Sehr hoch	Entfällt
Aktualität	Manuell sicherzustellen	Automatisch durch Anbieter
Datenmenge	Sehr groß	Nicht relevant
API-Kontingent	Nicht relevant	Begrenzt je nach Tarif
Implementierungs-Aufwand	Sehr hoch	Gering

Tabelle 2: Vergleich: Eigene Datenhaltung vs. externe API

5.9.3 Auswahlkriterien für eine Film-API

Für die Auswahl einer geeigneten Film-API wurden mehrere Kriterien definiert, die für die Anforderungen der vorliegenden Anwendung relevant sind:

- **Verfügbarkeit und Zugänglichkeit**: Die API soll über einen kostenlosen Zugang verfügen, der für Entwicklungs- und Testzwecke ausreichende Aufrufkontingente bietet.
- **Aktualität des Filmbestands**: Neue Filme sollen zeitnah in der Datenbank erfasst werden, sodass die Anwendung stets aktuelle Inhalte anzeigen kann.
- **Streaming-Provider-Integration**: Die API soll nicht nur Filmdaten, sondern auch Informationen zu Streaming-Verfügbarkeiten bereitstellen – also auf welchen Plattformen ein Film verfügbar ist. Dies ist eine Kernanforderung der Anwendung.
- **Provider-Gesamtliste**: Zusätzlich zur filmspezifischen Verfügbarkeit soll die API eine vollständige Liste aller bekannten Streaming-Anbieter bereitstellen, die in der eigenen Datenbank synchronisiert werden kann.
- **Dokumentationsqualität**: Eine verständliche und vollständige API-Dokumentation ist Voraussetzung für eine effiziente Integration.
- **Verbreitung und Community**: Eine weit verbreitete API bietet erfahrungsgemäß eine stabilere Schnittstelle, regelmäßige Updates und eine aktive Entwickler-Community.

Im Rahmen der Evaluierung wurden unter anderem die *Open Movie Database API (OMDb)* sowie die *TMDB API (The Movie Database)* betrachtet. Die OMDb API bietet grundlegende Filmdaten, verfügt jedoch über keine integrierte Bereitstellung von

Streaming-Verfügbarkeiten, was eine Kernanforderung der Anwendung darstellt. Damit schied diese Option aus.

5.9.4 Entscheidung für die TMDB API

Die Entscheidung fiel auf die **TMDB API (The Movie Database)**, da sie alle definierten Auswahlkriterien erfüllt. Die TMDB ist eine der größten öffentlich zugänglichen Film- und Seriendatenbanken und wird von einer aktiven Community kontinuierlich gepflegt. Der kostenlose API-Zugang bietet ausreichende Aufrufkontingente für den Betrieb der Anwendung.

Für die vorliegende Anwendung sind insbesondere folgende Endpunkte der TMDB API relevant:

- `/movie/{id}` – Abrufen von Filmdaten anhand einer ID
- `/search/movie` – Suche nach Filmen anhand eines Titels
- `/trending/movie/{timewindow}` – Abrufen trendender Filme
- `/movie/{id}/watch/providers` – Streaming-Anbieter eines Films, gefiltert nach Land
- `/watch/providers/movie` – Vollständige Liste aller bekannten Streaming-Anbieter

Besonders hervorzuheben ist die Verfügbarkeit des `/watch/providers`-Endpunkts, der nicht nur angibt, auf welchen Plattformen ein Film verfügbar ist, sondern auch zwischen Flatrate-, Leih- und Kaufangeboten unterscheidet. In der vorliegenden Anwendung werden ausschließlich Flatrate-Anbieter berücksichtigt, da der Fokus auf der Verwaltung aktiver Abonnements liegt. Die Authentifizierung gegenüber der TMDB API erfolgt über einen Bearer-Token, der über den Quarkus-Konfigurationsmechanismus mittels `@ConfigProperty` sicher injiziert wird und nicht im Quellcode hinterlegt ist.

6 Umsetzung der Anwendung

6.1 Implementierung der Abo-Verwaltung

Ein wesentlicher Bestandteil meines Projektteils war die Umsetzung der Benutzeroberfläche zur Verwaltung persönlicher Streaming-Abonnements. Ziel war es, eine übersichtliche und einfach bedienbare Ansicht zu entwickeln, in der Nutzende ihre aktiven Anbieter verwalten und Änderungen direkt speichern können. Die technische Umsetzung erfolgte im Angular-Frontend mit einer eigenständigen Komponente für die Providerverwaltung sowie zugehörigen Services für die Kommunikation mit dem Backend [4, 8].

6.1.1 Aufbau der Provider-Verwaltung

Die Provider-Verwaltung wurde als eigener geschützter Bereich innerhalb der Angular-Anwendung umgesetzt. Nutzende können dort Streaming-Anbieter laden, durchsuchen und auswählen. Zusätzlich ist erkennbar, welche Anbieter bereits im Profil gespeichert sind. Diese Struktur verbessert die Übersichtlichkeit und ermöglicht eine klare Trennung zwischen allgemeinen Filmfunktionen und persönlichen Abo-Daten.

Technisch wurde dieser Bereich als eigenständige Frontend-Ansicht mit zugehöriger Verwaltungslogik aufgebaut. Eine zuständige Angular-Komponente übernimmt die Darstellung der Anbieterübersicht, der aktiven Filter und der Eingabefelder für Abo-Daten. Der eigentliche Datenzugriff erfolgt über Services, die beim Öffnen der Ansicht zunächst die verfügbaren Anbieter sowie die bereits gespeicherten benutzerspezifischen Abonnements laden. Auf dieser Grundlage kann das Frontend sofort sichtbar machen, welche Anbieter bereits aktiv sind und bei welchen Einträgen noch keine persönlichen Daten hinterlegt wurden.

6.1.2 Bearbeitung und Speicherung von Abonnements

In der Oberfläche können Nutzende Anbieter als aktiv markieren und zusätzliche Angaben wie Abrechnungszyklus und Datum der letzten Abbuchung festlegen. Diese

Eingaben werden im Frontend verarbeitet und anschließend gesammelt an das Backend übermittelt. Die Verwendung von Formularlogik und Datenbindung in Angular erleichtert dabei die direkte Verknüpfung zwischen Eingabefeldern und angezeigten Daten [8]. Dadurch konnten Änderungen in der Oberfläche unmittelbar sichtbar gemacht werden.

Für die Umsetzung bedeutete das konkret, dass Änderungen zunächst im Zustand der Komponente verarbeitet und anschließend über einen zuständigen Service an das Backend übertragen werden. Die Oberfläche muss dabei sowohl bereits gespeicherte Daten korrekt anzeigen als auch neue oder geänderte Werte übernehmen. Dadurch entsteht ein durchgängiger Ablauf aus Laden, Bearbeiten, Absenden und erneuter Anzeige der aktualisierten Daten. Gerade bei der Abo-Verwaltung war diese Logik wichtig, weil nicht nur einzelne Werte angezeigt werden, sondern Anbieterstatus, Zyklus, Datumsangaben und weitere Abo-Metadaten fachlich zusammengehören.

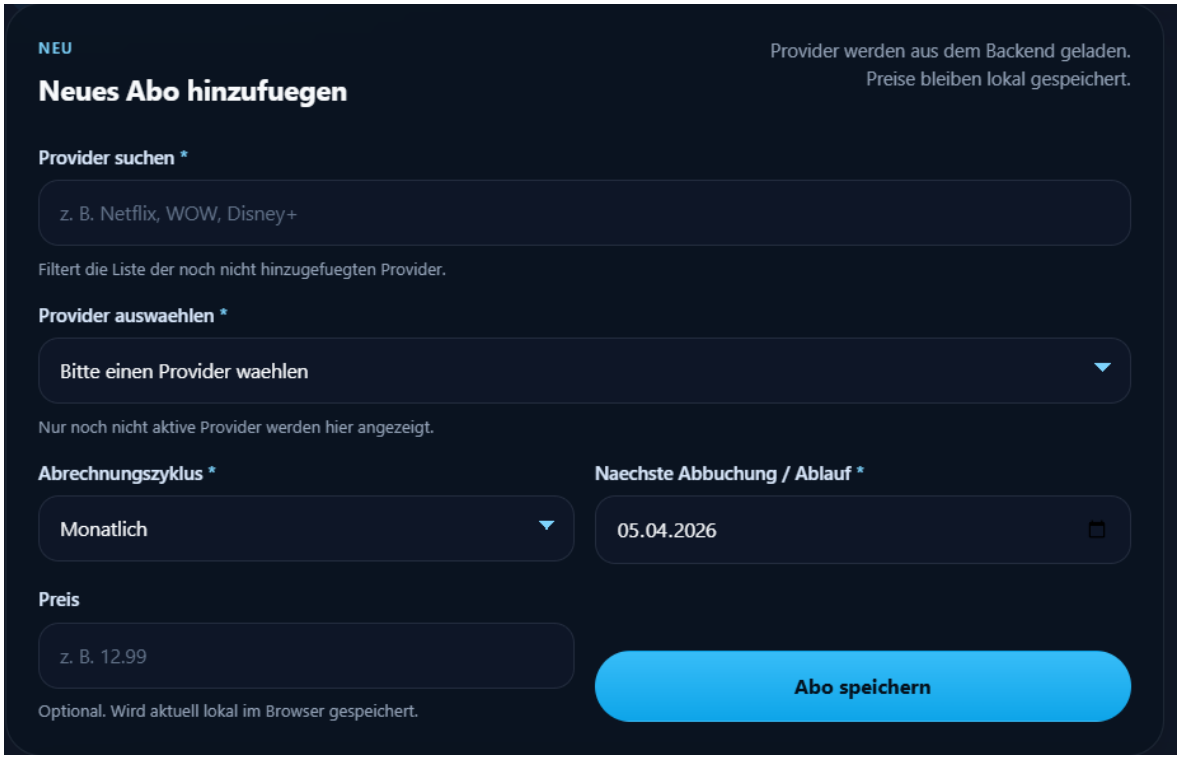

Abbildung 5: Erfassungsmaske für ein Streaming-Abonnement

6.1.3 Filter- und Suchfunktionen in der Oberfläche

Damit die Verwaltung vieler Anbieter benutzerfreundlich bleibt, wurde die Oberfläche um Such- und Filterfunktionen ergänzt. Nutzende können nach Anbietern suchen oder sich nur bereits aktivierte Abonnements anzeigen lassen. Diese Interaktionsmöglichkeiten

verbessern die Nutzbarkeit der Anwendung deutlich, da relevante Einträge schneller gefunden und bearbeitet werden können.

Die Such- und Filterlogik wurde direkt in der Benutzeroberfläche verarbeitet. Dazu werden geladene Anbieterlisten im Frontend anhand der aktuellen Eingaben gefiltert und anschließend in der Ansicht aktualisiert dargestellt. Dieses Vorgehen ist für den Projektkontext sinnvoll, weil die eigentlichen Abo-Daten bereits geladen sind und die Oberfläche schnell auf Änderungen reagieren soll, ohne bei jeder kleinen Filteranpassung einen neuen Request auszulösen. Die Kombination aus geladener Datenbasis, sichtbarer Markierung aktiver Abos und lokaler Suchlogik machte die Provider-Verwaltung deutlich alltagstauglicher.

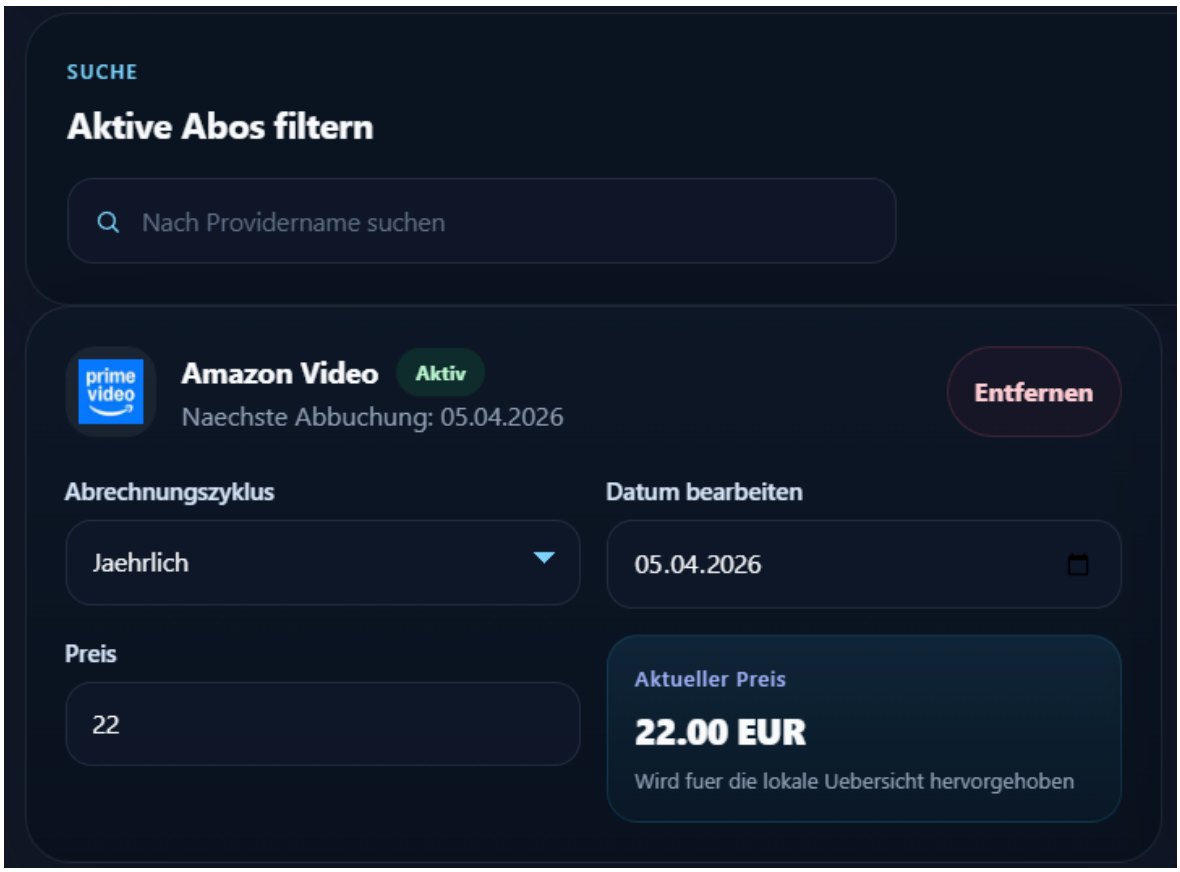

Abbildung 6: Abo-Verwaltung mit Such-, Filter- und Bearbeitungsfunktionen

6.2 Darstellung von Laufzeiten und Kündigungszeitpunkten

Neben der Auswahl von Streaming-Anbietern war auch die Darstellung zeitlicher Informationen ein wichtiger Teil der Abo-Verwaltung. Aus Frontend-Sicht ging es dabei

vor allem darum, Eingabemöglichkeiten für abrechnungsrelevante Daten bereitzustellen und diese verständlich in die Oberfläche einzubinden. Nutzende sollen dadurch ihre gespeicherten Abonnements besser nachvollziehen können.

6.2.1 Erfassung abrechnungsrelevanter Daten

Für jedes aktive Abonnement können im Frontend Informationen wie Abrechnungszyklus und Datum der letzten Abbuchung erfasst werden. Diese Werte werden in Formularfeldern dargestellt und von den Nutzenden bearbeitet. Die Oberfläche unterstützt dadurch nicht nur die Auswahl eines Anbieters, sondern auch die strukturierte Verwaltung der wichtigsten Abo-Metadaten. Bereits umgesetzt ist damit die Eingabe und Speicherung jener Daten, die später als Grundlage für weiterführende zeitliche Auswertungen dienen können.

6.2.2 Grenzen des aktuellen Umsetzungsstandes

Im aktuellen Projektstand liegt der Schwerpunkt klar auf der Erfassung und Speicherung dieser Daten. Noch nicht umgesetzt ist eine vollständige Berechnungslogik, die auf Basis dieser Eingaben automatisch konkrete Kündigungszeitpunkte, Erinnerungen oder bevorstehende Abbuchungen ableitet. Der Abschnitt beschreibt daher bewusst keinen bereits fertigen Fristenrechner, sondern den tatsächlichen Zwischenstand: Die notwendigen Eingabedaten können erfasst werden, weiterführende zeitliche Auswertung ist fachlich vorbereitet, aber noch kein abgeschlossener Kernbestandteil des Frontends.

Zukunftsthema bleiben damit insbesondere automatische Hinweise, Benachrichtigungen und eine stärker ausformulierte Darstellung kommender Kündigungs- oder Verlängerungszeitpunkte. Für die Bewertung des Projektstands ist diese Trennung wichtig, weil die aktuelle Umsetzung bereits einen verwertbaren Eingabestand bietet, die eigentliche Mehrwertlogik in diesem Teil jedoch noch nicht vollständig realisiert wurde.

6.3 Verknüpfung von Filmen mit Streaming-Anbietern

Ein zentraler Mehrwert der Anwendung besteht darin, dass Filmrecherche und Abo-Verwaltung in einer Oberfläche zusammengeführt werden. Im Frontend wurde dafür eine Benutzeroberfläche entwickelt, in der Nutzende Filme suchen, Trending-Inhalte

ansehen und Detailinformationen zu einzelnen Filmen aufrufen können. Die Verknüpfung mit Streaming-Anbietern ermöglicht es, Filminformationen nicht isoliert, sondern im Zusammenhang mit den eigenen Abonnements zu betrachten.

6.3.1 Umsetzung der Filmsuche

Die Filmsuche wurde als eigener Bereich im Frontend umgesetzt. Nutzende können nach Filmtiteln suchen und erhalten eine Liste passender Ergebnisse, die dynamisch geladen und angezeigt werden. Da das Frontend als Single-Page-Application realisiert wurde, bleiben Suchanfragen und Seitenwechsel innerhalb derselben Anwendung und wirken dadurch deutlich flüssiger [9, 5].

Technisch wird die Suchanfrage aus der zuständigen Suchkomponente an einen Angular-Service übergeben, der den Backend-Request auslöst. Das Backend ruft die externen Filmdaten ab, reduziert sie auf die benötigten Informationen und sendet die Ergebnisse an das Frontend zurück. Dort werden die Daten in einer Ergebnisliste dargestellt und können von Nutzenden direkt ausgewählt werden. Diese Struktur war für das Projekt wichtig, weil damit Suche, Ergebnisanzeige und spätere Weiterverarbeitung in der Detailansicht klar miteinander verbunden werden konnten.

6.3.2 Darstellung von Trending-Filmen

Zusätzlich zur klassischen Suche wurde eine Ansicht für Trending-Filme integriert. Dadurch können Nutzende aktuelle oder populäre Inhalte entdecken, ohne gezielt nach einem Titel suchen zu müssen. Diese Funktion erweitert die Anwendung um einen explorativen Aspekt und erhöht den praktischen Nutzen des Frontends.

6.3.3 Detailansicht einzelner Filme

Für einzelne Filme wurde eine eigene Detailansicht umgesetzt. Dort werden weiterführende Informationen wie Titel, Beschreibung, Erscheinungsjahr, Laufzeit, Genres und weitere Metadaten dargestellt. Die Aufbereitung dieser Daten im Frontend war wichtig, damit Nutzende nicht nur Suchergebnisse sehen, sondern Inhalte auch genauer beurteilen können.

Nach Auswahl eines Suchergebnisses oder eines Trending-Eintrags wird die jeweilige Film-ID an die Detailansicht übergeben. Diese Ansicht löst anschließend gezielt weitere Requests aus, um zusätzliche Filmdaten und die zugehörigen Anbieterinformationen zu laden. Dadurch bleibt die Suchansicht schlank, während die Detailseite genau die Informationen nachlädt, die für einen ausgewählten Film wirklich benötigt werden. Diese Trennung war für die Frontend-Struktur sinnvoll, weil Übersicht und Detaildarstellung unterschiedliche Anforderungen an Datenmenge und Darstellungstiefe besitzen.

6.3.4 Verbindung zur Anbieteranzeige

Ein besonderer Fokus lag auf der Darstellung von Streaming-Anbietern im Zusammenhang mit Filminformationen. Dadurch kann die Anwendung anzeigen, auf welchen Plattformen ein Film verfügbar ist. Aus Sicht des Frontends bestand die Aufgabe vor allem darin, diese Informationen übersichtlich darzustellen und sinnvoll in die Filmansichten zu integrieren. Diese Verbindung zwischen Inhalt und Verfügbarkeit stellt den eigentlichen Kernnutzen des Projekts dar.

In der Detailansicht werden Anbieter nicht nur allgemein aufgelistet, sondern nach Verfügbarkeitskategorien wie etwa Flatrate, Kaufen oder Leihen strukturiert angezeigt. Dadurch entsteht eine fachlich aussagekräftigere Darstellung, weil Nutzende sofort erkennen können, ob ein Film im Rahmen eines bestehenden Abos verfügbar ist oder nur kostenpflichtig zusätzlich erworben werden müsste. Gleichzeitig wird im Frontend ein Abgleich mit den gespeicherten eigenen Abonnements vorgenommen. Anbieter, die bereits in der persönlichen Abo-Verwaltung hinterlegt sind, können dadurch in der Detailansicht erkennbar hervorgehoben oder als bereits vorhanden kenntlich gemacht werden.

Gerade dieser Abgleich bildet den Kernnutzen der Anwendung: Nicht nur die Frage, wo ein Film theoretisch verfügbar ist, sondern ob er mit den bereits vorhandenen eigenen Abos angesehen werden kann. Die Verknüpfung von externer Filminformation und persönlicher Abo-Situation unterscheidet die Anwendung von einer reinen Filmrecherche oder einer isolierten Abo-Verwaltung und macht den fachlichen Mehrwert des Projekts unmittelbar sichtbar.

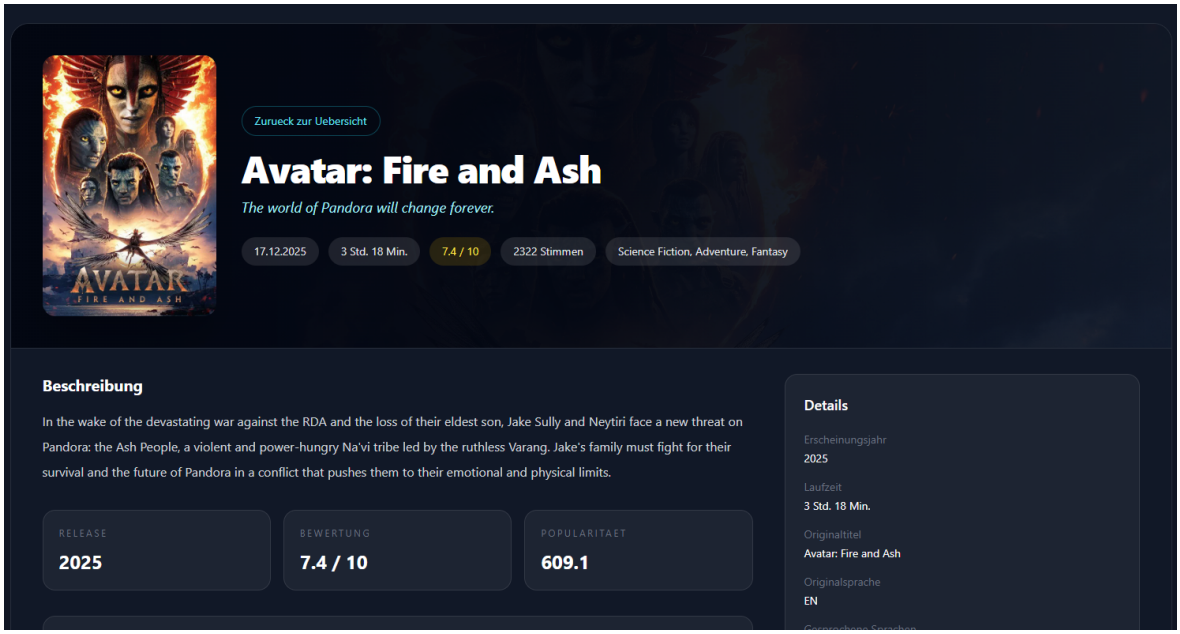

Abbildung 7: Film-Detailansicht mit Poster, Metadaten und Kurzbeschreibung

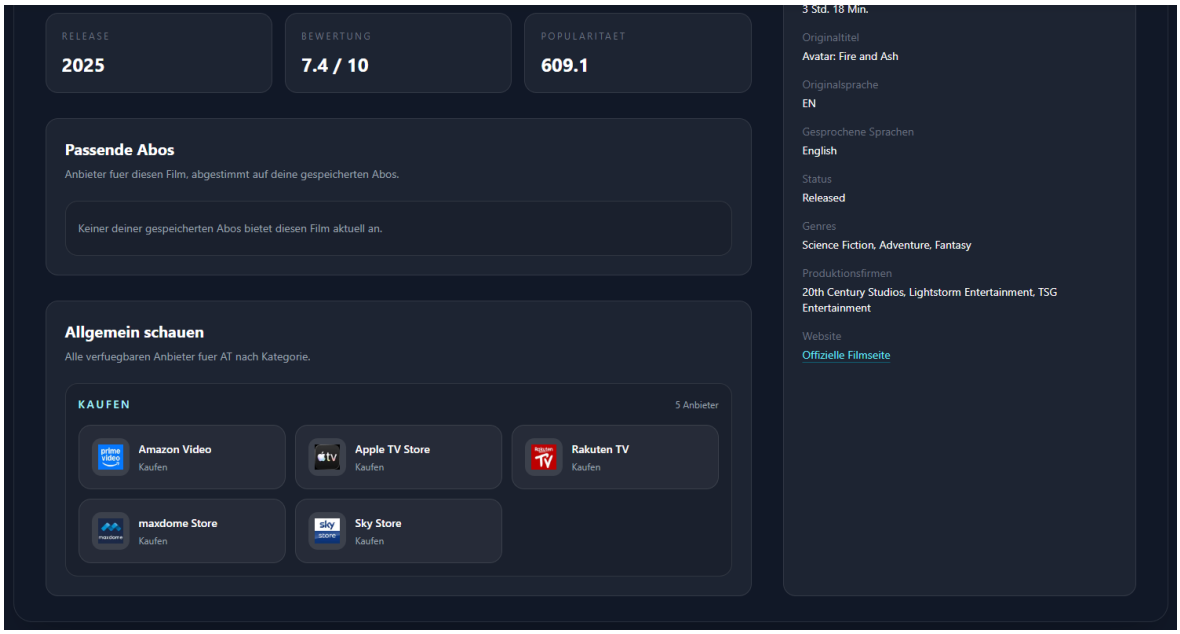

Abbildung 8: Film-Detailansicht mit Abgleich gespeicherter Abonnements und Anbieterübersicht

6.4 Account- und Passwortverwaltung

Die Benutzerverwaltung wurde im Frontend nicht als Eigenlösung umgesetzt, sondern über die Integration von Keycloak realisiert. Dadurch konnte auf eine eigene Passwortspeicherung verzichtet werden, während gleichzeitig ein professioneller und sicherer Anmeldeprozess in die Anwendung eingebunden wurde [12, 13]. Für meinen Projektteil

war vor allem relevant, wie sich Login, Registrierung und geschützte Routen in die Angular-Anwendung integrieren lassen.

6.4.1 Einbindung von Login und Registrierung

Im Frontend wurden Login- und Registrierungsfunktionen so eingebunden, dass Nutzende bei Bedarf an das Keycloak-System weitergeleitet werden. Nach erfolgreicher Anmeldung kehren sie in die Anwendung zurück und können geschützte Bereiche verwenden. Dadurch bleibt der Authentifizierungsprozess klar vom restlichen Frontend getrennt und dennoch benutzerfreundlich in die Anwendung eingebettet.

6.4.2 Geschützte Bereiche im Frontend

Bestimmte Bereiche, insbesondere die Verwaltung persönlicher Abonnements, sind nur für angemeldete Nutzende zugänglich. Das Frontend prüft dafür den Authentifizierungsstatus und steuert den Zugriff auf geschützte Routen. Dadurch wird verhindert, dass persönliche Funktionen ohne gültige Anmeldung aufgerufen werden können. Projektbezogen bedeutete das, dass öffentliche Filmfunktionen und persönliche Verwaltungsansichten bewusst unterschiedlich behandelt wurden: Filmsuche und Detailseiten bleiben offen nutzbar, während die Abo-Verwaltung erst nach erfolgreicher Anmeldung eingebunden wird.

Der Login-Status wirkt sich dabei auch auf die Navigation aus. In der Benutzeroberfläche musste erkennbar sein, ob eine Person bereits angemeldet ist und ob geschützte Menüpunkte verfügbar gemacht werden dürfen. Diese Kopplung zwischen Authentifizierungsstatus und Navigation war ein wichtiger Teil der Frontend-Logik, weil die Anwendung sonst entweder zu viele Funktionen öffentlich anzeigen oder geschützte Bereiche unklar behandeln würde.

6.4.3 Verwendung von Bearer-Token bei geschützten Requests

Ein wichtiger technischer Punkt der Umsetzung war, dass benutzerspezifische Daten nicht ohne Authentifizierung geladen werden können. Wenn das Frontend persönliche Informationen vom Backend anfordert, muss ein gültiges Bearer-Token mitgesendet werden. Dieses Token wird nach erfolgreicher Anmeldung über Keycloak bereitgestellt und

bei geschützten HTTP-Anfragen übertragen [12]. Erst dadurch kann das Backend prüfen, ob die anfragende Person berechtigt ist, auf die jeweiligen Daten zuzugreifen.

Für das Projekt war dabei vor allem entscheidend, diese Token-Verwendung nicht in jeder einzelnen Komponente separat zu behandeln. Die Authentifizierungslogik musste zentral organisiert werden, damit geschützte Requests für Abo-Daten, Änderungen und persönliche Anbieterinformationen konsistent verarbeitet werden können. Dadurch blieb die eigentliche Oberflächenlogik schlanker, während die sicherheitsrelevante Weitergabe des Tokens an zentraler Stelle mitgedacht werden konnte.

6.4.4 Herausforderungen bei der Integration von Keycloak

Die Integration von Keycloak in Angular war mit mehreren Herausforderungen verbunden. Dazu gehörten insbesondere die korrekte Weiterleitung nach dem Login, die Behandlung geschützter Routen sowie die sichere Übergabe des Bearer-Tokens bei Anfragen an das Backend. Diese Punkte waren im Frontend besonders relevant, da die Anwendung nur dann benutzerfreundlich funktioniert, wenn Anmeldung, Rückleitung und Zugriffsschutz zuverlässig zusammenspielen. Die Auseinandersetzung mit diesen Problemen war daher ein wichtiger Teil meines praktischen Projektbeitrags.

6.4.5 Analyse bestehender Lösungen

Im Zusammenhang mit Benutzerkonten stellte sich die Frage, ob zusätzlich eine eigene Passwortverwaltung innerhalb der Anwendung umgesetzt werden sollte. Bei der Betrachtung bestehender Lösungen wie Bitwarden oder 1Password wird jedoch deutlich, dass sich professionelle Passwortverwaltung auf Sicherheit, Verschlüsselung und geräteübergreifende Synchronisation spezialisiert [13]. Diese Zielsetzung unterscheidet sich klar vom Schwerpunkt des MedienAbonnementManagers.

6.4.6 Sicherheits- und Datenschutzbetrachtung

Eine selbst entwickelte Verwaltung von Passwörtern oder besonders sensiblen Zugangsdaten hätte hohe Anforderungen an Datenschutz und Sicherheit gestellt. Da der Fokus meines Projektteils auf der Frontend-Entwicklung lag, wurde bewusst auf eine eigene Passwortlösung verzichtet. Stattdessen war es sinnvoller, ein bestehendes Authentifizierungssystem einzubinden und die Anwendung auf ihre eigentlichen Kernfunktionen

zu konzentrieren. Die Entscheidung zugunsten von Keycloak war damit eine bewusste Abwägung zwischen Funktionsumfang, Sicherheit, Wartbarkeit und verfügbarem Projektzeitraum.

6.4.7 Entscheidung: Eigenlösung vs. Integration

Die Entscheidung fiel daher zugunsten einer Integration von Keycloak und gegen eine selbst entwickelte Passwortverwaltung. Eine Eigenlösung hätte zwar mehr direkten Gestaltungsspielraum geboten, hätte aber gleichzeitig zusätzliche Verantwortung für Passwortspeicherung, Sicherheitsprüfungen und Fehlerbehandlung bedeutet. Die gewählte Integration reduziert die Komplexität im Frontend, verbessert die Sicherheit und ermöglicht eine klarere Trennung zwischen Benutzeroberfläche und Authentifizierungslogik. Für das Projekt war diese Entscheidung sowohl technisch als auch organisatorisch sinnvoll.

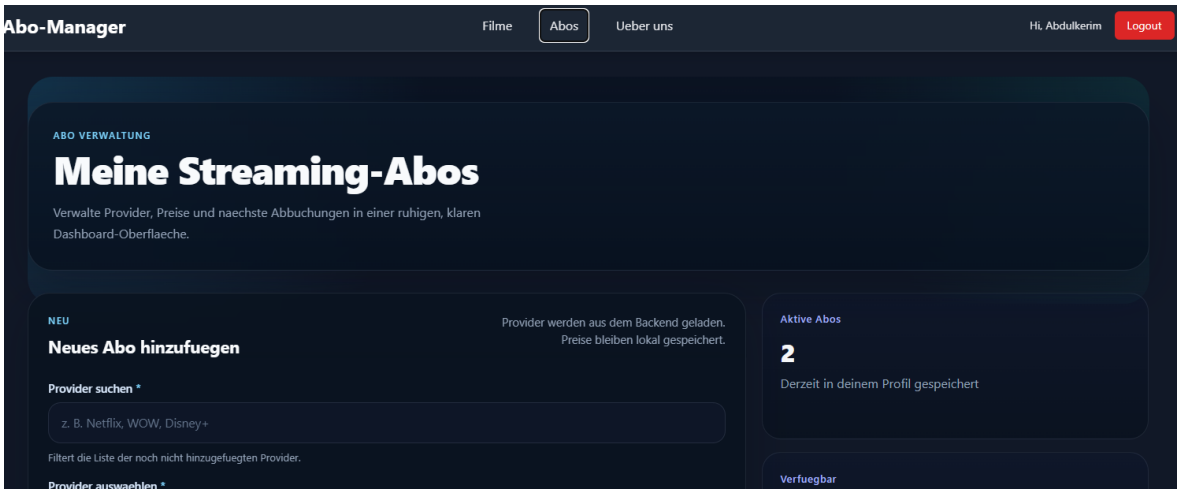

Abbildung 9: Geschützter Bereich der Abo-Verwaltung nach erfolgreicher Anmeldung

6.5 Herausforderungen und Reflexion der Frontend-Entwicklung

Im Verlauf der Frontend-Umsetzung zeigte sich, dass nicht nur die reine Darstellung von Oberflächen relevant ist, sondern vor allem die saubere Verbindung zwischen Benutzerführung, Authentifizierung und Datenlogik. Gerade dort entstanden mehrere Erkenntnisse, die für den Projektverlauf und für die eigene fachliche Weiterentwicklung wesentlich waren.

6.5.1 Herausforderungen bei der Integration von Keycloak in Angular

Eine der größten Herausforderungen meines Projektteils war die Integration von Keycloak in das Angular-Frontend. Ein praktisches Problem dabei war, dass für aktuelle Angular-Versionen vergleichsweise wenig gut strukturierte und direkt übertragbare Beispiele verfügbar waren. Viele gefundene Lösungen bezogen sich auf ältere Projektstände, unvollständige Konfigurationen oder andere Framework-Kombinationen. Dadurch war es notwendig, die Einbindung von Login, Redirect, Token-Verwaltung und geschützten Routen schrittweise selbst nachzuvollziehen und an die eigene Anwendung anzupassen.

Ein konkretes Beispiel war die Frage, wann geschützte Inhalte tatsächlich geladen werden dürfen. Die reine erfolgreiche Anmeldung reichte dafür noch nicht aus; zusätzlich musste sichergestellt werden, dass der Rückweg aus dem Login korrekt verarbeitet wird und geschützte Requests erst dann abgesendet werden, wenn der Authentifizierungsstatus und das Token im Frontend wirklich verfügbar sind. Diese Abstimmung zwischen Routing, Login-Rückleitung und Datenabruf war kein rein theoretisches Problem, sondern eine praktische Hürde in der Umsetzung.

6.5.2 Herausforderungen bei der Umsetzung von Backend-Daten in der Benutzeroberfläche

Ein weiterer anspruchsvoller Punkt war die Übersetzung der vom Backend gelieferten Informationen in eine für Nutzende logische und leicht verständliche Oberfläche. Die gelieferten Daten mussten nicht nur technisch korrekt angezeigt, sondern auch sinnvoll gruppiert, gefiltert und mit Interaktionen verknüpft werden. Besonders bei Providerdaten, persönlichen Abonnements und filmbasierten Anbieterinformationen war es wichtig, die Datenstruktur aus Services und DTOs so in die UI zu übertragen, dass die Oberfläche konsistent bleibt und Nutzende die Zusammenhänge schnell erfassen können.

Ein konkretes Beispiel dafür war die Detailansicht eines Films mit Anbieterabgleich. Das Backend liefert einerseits allgemeine Anbieterinformationen zum ausgewählten Film und andererseits benutzerspezifische Abo-Daten. Im Frontend mussten diese beiden Datenquellen so zusammengeführt werden, dass sichtbar wird, welche Anbieter

grundsätzlich vorhanden sind und welche davon bereits zum persönlichen Profil gehören. Die Herausforderung lag also nicht nur im Anzeigen der Daten, sondern im sinnvollen Zusammenführen zweier fachlich unterschiedlicher Informationsarten innerhalb derselben Ansicht.

6.5.3 Reflexion der anfänglichen Frontend-Entscheidungen

Rückblickend wurde deutlich, dass eine frühe Oberflächenidee nicht automatisch auch die beste Struktur für die spätere Weiterentwicklung darstellt. Zu Beginn lag der Fokus stark darauf, einzelne Funktionen möglichst schnell sichtbar zu machen. Erst im Verlauf der Umsetzung zeigte sich, wie wichtig eine klare Trennung zwischen Komponenten, Services, Zuständen und geschützten Bereichen ist. Diese Erkenntnis war für die Frontend-Entwicklung ähnlich wertvoll wie die Datenbank-Reflexion im Backend, weil sie gezeigt hat, dass gute Wartbarkeit nicht nur durch funktionierenden Code entsteht, sondern durch eine durchdachte Oberflächen- und Zustandsstruktur. Aus heutiger Sicht würde ich diese Struktur früher festlegen und die Datenflüsse von Beginn an konsequenter entlang fachlicher Verantwortlichkeiten aufteilen.

Ein Beispiel dafür war die anfängliche Tendenz, Datenlogik zu nah an sichtbaren Komponenten zu behandeln. Solange einzelne Ansichten noch klein waren, funktionierte dieser pragmatische Ansatz, mit wachsender Komplexität bei Suche, Detaildarstellung und Abo-Verwaltung wurde er jedoch unübersichtlicher. Erst die stärkere Verlagerung von Request-Logik und Datenverarbeitung in Services machte die Oberfläche langfristig wartbarer.

6.5.4 Erkenntnisgewinn für den weiteren Projektverlauf

Die Auseinandersetzung mit diesen Frontend-Problemen hat wesentlich dazu beigetragen, die Anwendung systematischer weiterzuentwickeln. Insbesondere die Keycloak-Integration und die saubere Abbildung von Backend-Daten auf UI-Komponenten haben gezeigt, dass moderne Webentwicklung nicht nur aus Design und API-Aufrufen besteht, sondern ein durchgängiges Verständnis von Authentifizierung, Datenfluss und Benutzerführung erfordert. Gerade dieser Lernprozess stellt einen wichtigen Teil meines persönlichen Beitrags zur Diplomarbeit dar.

7 Projektverlauf und Entscheidungen

7.1 Projektplanung und Arbeitsweise

Die Umsetzung des Projekts erfolgte arbeitsteilig innerhalb des Teams. Mein Schwerpunkt lag auf dem Frontend, während mein Kollege die backendseitigen Grundlagen, die Datenhaltung und die serverseitige Logik übernahm. Diese Aufteilung war für das Projekt sinnvoll, da sich sowohl die technischen Anforderungen als auch die Entwicklungsaufgaben deutlich zwischen Benutzeroberfläche und serverseitiger Verarbeitung unterschieden. Gleichzeitig war es notwendig, beide Bereiche regelmäßig aufeinander abzustimmen, damit Datenmodelle, Schnittstellen und Benutzerführung konsistent zusammenarbeiten konnten.

Zu Beginn des Projekts wurde eine To-do-Liste erstellt, um die wichtigsten Funktionen und Arbeitsschritte zu strukturieren. Diese frühe Planung half dabei, den Umfang des Projekts greifbar zu machen und erste Schwerpunkte festzulegen. Im weiteren Verlauf wurde die Arbeitsweise jedoch informeller. Verbesserungen, neue Anforderungen und technische Anpassungen ergaben sich häufig direkt aus der laufenden Umsetzung. Dadurch entstand kein streng linearer Ablauf, sondern ein schrittweiser Entwicklungsprozess, bei dem Planung und Umsetzung eng miteinander verbunden waren.

Die Abstimmung im Team erfolgte unterschiedlich, je nach aktueller Arbeitsphase. Da ein persönlicher Austausch in der Schule häufig möglich war, konnten viele Fragen direkt vor Ort geklärt werden. Ergänzend dazu wurde online per Chat kommuniziert, insbesondere wenn technische Probleme, Schnittstellenfragen oder konkrete Detailentscheidungen zwischen Frontend und Backend abgestimmt werden mussten. Diese Mischung aus direkter und digitaler Zusammenarbeit erwies sich für das Projekt als praktikabel, da sie kurze Rückkopplungen ermöglichte, ohne dass ein komplexes Projektmanagement-Werkzeug erforderlich war.

Auch die Entwicklung im Git-Repository zeigt diesen schrittweisen Projektverlauf. Frühere Commits verweisen zunächst auf grundlegende UI- und Navigationsarbeiten wie die Suchfunktion, das Styling oder Login- und Registeransätze. Darauf aufbauend

folgten Phasen, in denen zentrale Funktionen wie Trending-Filme, Abo-Verwaltung und Keycloak-Anbindung umgesetzt wurden. Spätere Commit-Nachrichten wie „Moviesearch überarbeitet und bessere Filterung eingebaut“, „Abo Verwaltung hinzugefügt und Bugfixes“ oder „Keycloak Frontend fertiggestellt“ machen deutlich, dass die Entwicklung nicht aus isolierten Einzelschritten bestand, sondern aus mehreren aufeinander aufbauenden Verbesserungsphasen.

7.2 Technische und fachliche Herausforderungen

Im Verlauf des Projekts traten mehrere Herausforderungen auf, die sowohl die technische Umsetzung als auch die fachliche Struktur der Anwendung betrafen. Eine zentrale Schwierigkeit bestand für mich darin, die Benutzeroberfläche nicht nur optisch umzusetzen, sondern sinnvoll mit der zugrunde liegenden Logik zu verbinden. Gerade bei der Abo-Verwaltung stellte sich die Frage, welche Informationen für Nutzende tatsächlich relevant sind, wie sie dargestellt werden sollen und an welcher Stelle Interaktion, Übersichtlichkeit und funktionale Tiefe im Gleichgewicht stehen.

Ein weiterer anspruchsvoller Punkt betraf die Kommunikation zwischen Frontend und Backend. Dabei ging es nicht nur darum, Daten technisch abrufen zu können, sondern auch darum, gemeinsam festzulegen, welche Informationen für das Projekt wirklich notwendig sind. Für das Frontend war diese Frage besonders wichtig, da nicht jede backendseitig verfügbare Information automatisch auch einen Mehrwert in der Benutzeroberfläche erzeugt. Die Auswahl relevanter Datenstrukturen, DTOs und Schnittstellen war daher ein wiederkehrender Abstimmungsprozess.

Zusätzlich ergaben sich technische Probleme durch Angular-Versionen und Änderungen innerhalb des Frameworks. Einzelne Implementierungen mussten angepasst oder überarbeitet werden, weil sich Verhaltensweisen, Bibliotheken oder Kompatibilitäten veränderten. Dadurch entstand stellenweise der Eindruck, dass funktionierender Code nicht dauerhaft stabil blieb, sondern nach Framework-Änderungen erneut überprüft werden musste. Diese Erfahrung war insbesondere im Frontend relevant, da moderne Webentwicklung stark von sich rasch weiterentwickelnden Technologien geprägt ist.

Eine weitere Herausforderung lag in der Keycloak-Integration. Zwar konnte die Authentifizierung grundsätzlich erfolgreich eingebunden werden, jedoch war die praktische

Verbindung zwischen Login, Token-Verwaltung, geschützten Routen und geschützten Backend-Requests mit mehreren Umsetzungsschritten verbunden. Die Schwierigkeit bestand hier weniger in einer einzelnen Funktion, sondern vielmehr im zuverlässigen Zusammenspiel mehrerer technischer Komponenten. Aus heutiger Sicht war gerade diese Herausforderung besonders lehrreich, weil sie gezeigt hat, wie eng Benutzerführung, Sicherheit und Datenzugriff in einer modernen Webanwendung miteinander verknüpft sind.

Ein konkretes Beispiel dafür war die Abstimmung zwischen Login-Rückleitung und geschützten Datenaufrufen. Erst wenn der Authentifizierungsstatus im Frontend korrekt verarbeitet war, konnten geschützte Requests für die Abo-Verwaltung sinnvoll ausgelöst werden. Wurde dieser Ablauf nicht sauber abgestimmt, bestand die Gefahr, dass persönliche Daten zu früh angefragt oder geschützte Ansichten zwar sichtbar, aber noch nicht korrekt mit Daten versorgt wurden.

Auch die Git-Entwicklung verdeutlicht diese Phase technischer Iteration. Commits wie „durchs Reskripten sind 3 Funktionen kaputt“, „es rennt jetzt, paar Funktionen wurde noch nicht implementiert aber großer Teil wurde umgescriptet“ oder „Filme id hinzugefügt und Abos Logik eingebaut, jedoch fehlt einiges und funktioniert noch nicht“ zeigen, dass der Projektverlauf nicht rein geradlinig war. Vielmehr mussten funktionierende Teilstände immer wieder überarbeitet, stabilisiert und weiterentwickelt werden. Gerade diese Zwischenphasen waren für den realen Entwicklungsprozess charakteristisch.

7.3 Entscheidungsfindung im Projektteam

Die Entscheidungsfindung im Team erfolgte situationsabhängig. Viele technische und fachliche Fragen wurden gemeinsam diskutiert, insbesondere dann, wenn sie Auswirkungen auf beide Projektteile hatten. Dazu zählten vor allem Fragen der Datenstruktur, der Schnittstellen zwischen Frontend und Backend sowie die Festlegung, welche Funktionen für das Projekt wirklich im Mittelpunkt stehen sollten. Durch diese Diskussionen konnten unterschiedliche Sichtweisen zusammengeführt werden, bevor konkrete Umsetzungen begonnen wurden.

Gleichzeitig arbeitete jeder im eigenen Verantwortungsbereich weitgehend selbstständig. In meinem Fall betraf dies vor allem Entscheidungen zur Gestaltung der Benutzeroberfläche, zur Struktur von Komponenten und zur Einbindung von Authentifizierung und

Benutzerinteraktion im Frontend. Obwohl die Umsetzung eigenständig erfolgte, wurde bei wichtigen Fragen regelmäßig Feedback eingeholt. Diese Arbeitsweise war sinnvoll, da sie einerseits Eigenverantwortung ermöglichte und andererseits verhinderte, dass Frontend und Backend inhaltlich auseinanderliefen.

Inhaltlich spielte auch die Abgrenzung des Projektumfangs eine wichtige Rolle. Ein Beispiel dafür war die Frage, welche Inhalte auf der Website tatsächlich angezeigt werden sollten. Im Verlauf der Abstimmung wurde unter anderem diskutiert, ob sich die Anwendung nur auf Filme oder zusätzlich auch auf Serien und Fernsehinhalte beziehen soll. Solche Entscheidungen waren nicht bloß Detailfragen, sondern beeinflussten unmittelbar die Ausrichtung der Anwendung und damit auch die technische Umsetzung. Rückblickend zeigte sich, dass die bewusste Eingrenzung auf Filme wesentlich dazu beitrug, den Projektfokus klar zu halten.

7.4 Einbindung von Betreuern und Feedback

Die Einbindung von Betreuern und Lehrpersonen war im Projekt vor allem dort hilfreich, wo eine Außenperspektive auf Prioritäten, Umfang und inhaltliche Ausrichtung benötigt wurde. Auch wenn nicht jede Projektentscheidung direkt von außen beeinflusst wurde, half das erhaltene Feedback dabei, unklare Punkte schneller zu konkretisieren. Besonders wichtig war dies in Situationen, in denen zwar bereits technische Möglichkeiten vorhanden waren, aber noch nicht eindeutig feststand, welche davon für die Diplomarbeit tatsächlich den größten Nutzen bringen würden.

Das Feedback bezog sich unter anderem auf die Frage, welche Inhalte im Projektkern stehen sollen und welche Funktionen für die Anwendung wirklich relevant sind. Gerade bei der Filmsuche und bei der Auswahl anzuzeigender Inhalte war diese Unterstützung hilfreich, da dadurch der Fokus der Anwendung geschärft wurde. Ebenso wurde deutlich gemacht, dass nicht jede theoretisch denkbare Erweiterung in den Projektumfang aufgenommen werden muss, wenn sie den Kern der Arbeit verwässern würde.

Aus meiner Sicht war diese Form der Einbindung besonders wertvoll, weil sie nicht nur auf technische Details abzielte, sondern vor allem bei der Priorisierung half. Damit wurde der Projektverlauf nicht grundlegend umgelenkt, aber an mehreren Stellen präzisiert. Genau diese punktuelle fachliche Lenkung war für ein Projekt mit mehreren möglichen Entwicklungspfaden hilfreich.

7.5 Reaktion auf Fragen und Kritik aus Präsentationen

Formale Zwischenpräsentationen mit umfangreicher Kritik im klassischen Sinn fanden im Projektverlauf nicht in stark strukturierter Form statt. Rückmeldungen kamen jedoch von Kollegen und aus dem erweiterten schulischen Umfeld, wodurch trotzdem eine wertvolle Außenperspektive entstand. Diese Rückmeldungen bezogen sich häufig auf praktische Fragen der Nutzung und Sicherheit, also genau auf jene Aspekte, die für eine anwendungsnahe Weblösung besonders relevant sind.

Ein Beispiel dafür war die Frage, welche Bereiche durch Keycloak geschützt werden sollten und welche Daten öffentlich zugänglich bleiben können. Solche Hinweise waren wichtig, weil sie halfen, die Grenze zwischen offenen Filmfunktionen und persönlicher Abo-Verwaltung klarer zu ziehen. Auch wenn diese Rückmeldungen nicht in Form formaler Meilenstein-Präsentationen stattfanden, hatten sie dennoch Einfluss auf die Präzisierung einzelner Funktionen.

Insgesamt zeigt sich, dass Kapitel 7 weniger durch starre Projektmanagement-Strukturen geprägt war, sondern stärker durch einen realen, iterativen Entwicklungsprozess. Planung, technische Umsetzung, Abstimmung und Feedback liefen nicht strikt getrennt voneinander ab, sondern beeinflussten sich gegenseitig. Gerade darin liegt ein wesentlicher Erkenntnisgewinn des Projekts: Die Entwicklung einer fachlich sinnvollen Anwendung besteht nicht nur aus dem Schreiben von Code, sondern ebenso aus wiederholtem Abgleichen von Anforderungen, Nutzen, technischer Machbarkeit und Benutzerperspektive.

8 Ergebnisse und Evaluation

8.1 Erfüllung der gesetzten Ziele

Kapitel 8 bewertet den erreichten Stand meines Frontend-Projektteils im Hinblick auf die ursprünglich verfolgten Ziele. Im Mittelpunkt stand dabei nicht die vollständige Abdeckung aller denkbaren Medienfunktionen, sondern die Entwicklung einer funktionalen und nachvollziehbaren Benutzeroberfläche für die zentrale Verwaltung persönlicher Streaming-Abonnements. Aus heutiger Sicht konnte dieses Kernziel in wesentlichen Teilen erreicht werden.

8.1.1 Erreichte Kernziele im Frontend

Ein zentrales Ziel meines Projektteils war die Umsetzung einer Benutzeroberfläche, in der Nutzende ihre Streaming-Abonnements verwalten können. Dieses Ziel wurde grundsätzlich erreicht. Die Anwendung bietet einen geschützten Bereich, in dem Anbieter geladen, gefiltert, ausgewählt und gemeinsam mit zusätzlichen Angaben wie Abrechnungszyklus oder Datum der letzten Abbuchung gespeichert werden können. Dadurch entstand eine Frontend-Lösung, die den Kernnutzen von Mediahub bereits sichtbar und praktisch nutzbar macht.

Auch die Integration der Authentifizierung mit Keycloak kann als erreichtes Ziel bewertet werden. Login, Registrierung, geschützte Routen und die Übergabe von Bearer-Tokens an das Backend wurden so umgesetzt, dass persönliche Funktionen nicht öffentlich zugänglich sind. Gerade für eine Anwendung, in der benutzerspezifische Abo-Daten verarbeitet werden, stellt dies einen wesentlichen Qualitätsfaktor dar. Die erfolgreiche Einbindung dieser Sicherheitslogik war daher nicht nur ein technisches Detail, sondern ein zentraler Bestandteil des Projektziels.

Ein weiteres wichtiges Ziel war die Verbindung von Filmsuche und Anbieterinformationen innerhalb einer gemeinsamen Oberfläche. Auch dieses Ziel wurde weitgehend erreicht. Nutzende können nach Filmen suchen, Detailinformationen aufrufen und erkennen, auf

welchen Plattformen ein Film verfügbar ist. Besonders relevant ist dabei die Verknüpfung mit den eigenen gespeicherten Abonnements, da dadurch nicht nur allgemeine Filmdaten angezeigt werden, sondern ein konkreter Mehrwert für die Nutzung entsteht.

8.1.2 Teilweise erreichte oder bewusst eingegrenzte Ziele

Trotz dieses insgesamt positiven Ergebnisses wurden nicht alle denkbaren Funktionen vollständig ausgebaut. Die Film-Detailseite ist funktional bereits gut nutzbar, weist jedoch in einzelnen Bereichen noch Potenzial für gestalterische oder inhaltliche Verfeinerungen auf. Dazu zählen kleinere Verbesserungen in der Darstellung, in der Informationsaufbereitung und in der weiteren Ausdifferenzierung einzelner UI-Elemente. Das Ziel einer funktionierenden Detailansicht wurde erreicht, die Ausarbeitung kann jedoch noch weiter verbessert werden.

Auch die Abo-Verwaltung ist im Kern funktionsfähig, wurde jedoch nicht in jedem Punkt vollständig zu Ende entwickelt. Die Erfassung und Speicherung der Daten funktioniert, dennoch wären zusätzliche Funktionen wie noch klarere Auswertungen, erweiterte Komfortfunktionen oder eine tiefergehende Unterstützung bei Kündigungs- und Abrechnungsfragen sinnvoll. Der aktuelle Stand bildet somit eine tragfähige Grundlage, ist aber noch nicht als vollständig ausgeschöpfte Endlösung zu verstehen.

Die Verlaufs- beziehungsweise History-Funktion ist ebenfalls ein Beispiel für einen nur teilweise erreichten Funktionsumfang. Sie wurde im Projekt berücksichtigt, aber nicht in dem Maß ausgebaut, dass daraus bereits ein gleichwertig ausgereifter Kernbestandteil der Anwendung geworden wäre. Im Rahmen der Bewertung ist daher wichtig, diese Funktion ehrlich als Teilergebnis einzuordnen.

8.1.3 Bewusste Reduktion des Projektumfangs

Für die Beurteilung der Zielerreichung ist außerdem wesentlich, dass einzelne Themen bewusst nicht in den Mittelpunkt aufgenommen wurden. Dazu zählen insbesondere Mediatheken, Livestreams oder sportbezogene Inhalte wie Fußball-Übertragungen. Diese Bereiche wären fachlich interessant gewesen, hätten den Umfang des Projekts jedoch deutlich vergrößert und die inhaltliche Klarheit geschwächt.

Die Entscheidung, den Fokus auf Filme zu legen, war daher aus heutiger Sicht sinnvoll. Sie ermöglichte es, eine klar abgegrenzte Anwendung umzusetzen, statt viele Themen

nur oberflächlich anzuschneiden. Die gesetzten Ziele wurden dadurch nicht verkleinert, sondern präzisiert. Gerade im Rahmen einer Diplomarbeit ist eine klar eingegrenzte, nachvollziehbar umgesetzte Lösung oft wertvoller als ein zu breites, aber unausgereiftes System.

8.2 Bewertung des Nutzens für Endanwender

Der praktische Nutzen der Anwendung ergibt sich vor allem aus der Zusammenführung zweier bislang getrennter Perspektiven: einerseits der persönlichen Verwaltung vorhandener Streaming-Abonnements und andererseits der Frage, wo ein bestimmter Film tatsächlich verfügbar ist. Für Endanwender entsteht dadurch ein Mehrwert, weil Informationen nicht mehr an mehreren Stellen getrennt gesucht und gedanklich zusammengeführt werden müssen.

8.2.1 Mehrwert durch zentrale Abo-Verwaltung

Die Abo-Verwaltung bietet für Nutzende vor allem deshalb einen konkreten Nutzen, weil sie eine zentrale Übersicht über gespeicherte Anbieter schafft. In der Praxis entstehen Probleme häufig nicht dadurch, dass einzelne Abonnements schwer abschließbar wären, sondern dadurch, dass im Laufe der Zeit mehrere Dienste parallel genutzt werden und der Überblick über Kosten, Laufzeiten und tatsächliche Verwendung verloren geht. Bereits die strukturierte Erfassung dieser Informationen kann dazu beitragen, Entscheidungen bewusster zu treffen und unnötige oder doppelte Abonnements eher zu erkennen.

Hinzu kommt, dass die Verwaltung persönlicher Anbieter innerhalb eines geschützten Bereichs stattfindet. Dadurch wird klar zwischen öffentlich verfügbaren Filmdaten und privaten Benutzerdaten unterschieden. Für Endanwender ist dies nicht nur technisch relevant, sondern auch vertrauensbildend, da die Anwendung erkennbar zwischen allgemeiner Recherche und personenbezogener Verwaltung trennt.

8.2.2 Mehrwert durch Filmsuche und Anbieterabgleich

Ein besonders hoher praktischer Nutzen ergibt sich aus der Filmrecherche in Verbindung mit Anbieterinformationen. Nutzende können nicht nur nach Filmen suchen, sondern direkt erkennen, welche Anbieter grundsätzlich verfügbar sind und welche davon bereits

zu den eigenen gespeicherten Abonnements gehören. Diese Verknüpfung reduziert den Aufwand, externe Plattformen einzeln zu prüfen, und unterstützt damit genau jenes Problem, das zur Motivation des Projekts beigetragen hat.

Gerade aus Frontend-Sicht war dieser Punkt besonders wichtig, weil die Anwendung dadurch nicht bloß Verwaltungsoberfläche bleibt, sondern einen unmittelbaren Alltagsnutzen erzeugt. Die Informationen werden so dargestellt, dass Nutzende nicht nur Daten abrufen, sondern daraus eine konkrete Entscheidung ableiten können, etwa ob ein Film bereits mit den vorhandenen Abos angesehen werden kann.

8.2.3 Grenzen des aktuellen Nutzens

Der Nutzen für Endanwender ist jedoch nicht in allen Bereichen gleich stark ausgeprägt. Da sich die Anwendung derzeit auf Filme konzentriert, fehlt eine breitere Abdeckung weiterer Medienarten. Ebenso sind Komfortfunktionen wie automatische Erinnerungen, weiterführende Kostenanalysen oder ein vollständig ausgebauter Verlauf noch nicht in einer Form vorhanden, die den täglichen Nutzen weiter erhöhen würde.

Daraus ergibt sich insgesamt ein differenziertes Bild: Der bestehende Projektstand bietet bereits einen erkennbaren und praktischen Mehrwert, insbesondere für Nutzende, die ihre Filmrecherche und ihre Streaming-Abos an einer Stelle zusammenführen möchten. Gleichzeitig zeigt die Anwendung deutlich, an welchen Stellen zukünftige Erweiterungen den Endanwendernutzen noch weiter steigern könnten.

8.3 Reflexion der eigenen Leistung

Die Bewertung der eigenen Leistung fällt insgesamt positiv aus, muss jedoch zugleich selbstkritisch bleiben. Mein Beitrag im Projekt lag klar im Frontend. Dort ging es nicht nur darum, Oberflächen optisch darzustellen, sondern eine funktionierende, geschützte und mit dem Backend abgestimmte Anwendung zu entwickeln. Gerade in dieser Verbindung aus UI, Datenfluss und Authentifizierung lag der eigentliche Anspruch meiner Arbeit.

8.3.1 Positiv bewertete Aspekte des eigenen Beitrags

Positiv bewerte ich vor allem, dass ich mehrere anspruchsvolle Frontend-Bestandteile tatsächlich umsetzen konnte. Dazu zählen die Filmsuche, die Film-Detailansicht, die geschützte Abo-Verwaltung sowie die Integration von Keycloak. Besonders die Keycloak-Anbindung war für mich ein wichtiger Lernschritt, weil sie weit über reine Oberflächengestaltung hinausging und ein tieferes Verständnis für Authentifizierung, geschützte Routen und den Umgang mit Tokens erforderte.

Ebenso positiv ist aus meiner Sicht, dass sich mein Projektteil im Laufe der Zeit deutlich weiterentwickelt hat. Anfangs stand stärker im Vordergrund, Funktionen überhaupt sichtbar und bedienbar zu machen. Mit zunehmendem Projektfortschritt wurde mir jedoch klarer, wie wichtig saubere Struktur, sinnvolle Datenübergabe und eine konsistente Benutzerführung sind. Dass ich diese Erkenntnisse nicht nur theoretisch gewonnen, sondern schrittweise in die Anwendung eingearbeitet habe, bewerte ich als wesentlichen persönlichen Fortschritt.

8.3.2 Selbstkritische Betrachtung und Verbesserungsmöglichkeiten

Selbstkritisch betrachtet hätte ich einzelne Frontend-Entscheidungen früher strukturierter vorbereiten können. Gerade bei der Verbindung zwischen Backend-Daten und Benutzeroberfläche zeigte sich, dass eine zunächst funktionierende Lösung nicht automatisch auch die klarste oder wartbarste Lösung ist. Teilweise mussten Komponenten, Datenflüsse oder Darstellungen im Nachhinein überarbeitet werden, weil erst im praktischen Einsatz deutlich wurde, welche Informationen wirklich relevant sind und wie sie für Nutzende logisch angeordnet sein sollten. Aus heutiger Sicht würde ich daher die fachlichen Anforderungen früher in konkrete Zuständigkeiten von Komponenten und Services übersetzen und die Authentifizierungslogik von Anfang an stärker als Querschnittsthema mitdenken.

Ein greifbares Beispiel war der Anbieterabgleich in der Film-Detailansicht. Dort mussten allgemeine Verfügbarkeitsdaten eines Films und die persönlich gespeicherten Abonnements in einer gemeinsamen Darstellung zusammengeführt werden. Erst im Verlauf der Umsetzung wurde deutlich, dass diese Ansicht nicht nur eine einfache Anzeige externer Daten ist, sondern eine präzise Abstimmung zwischen mehreren Services und einem klaren Zustandsmodell im Frontend erfordert.

Auch bei der Abo-Verwaltung und der Detailansicht ist aus heutiger Sicht erkennbar, dass noch Raum für Verfeinerungen besteht. Zwar sind die zentralen Funktionen vorhanden, doch zusätzliche Komfortfunktionen, eine noch klarere UX und eine weiter ausgebaute Unterstützung für zeitliche Abo-Aspekte würden die Qualität des Frontends weiter erhöhen. Die teilweise umgesetzte History-Funktion zeigt ebenfalls, dass nicht jeder begonnene Bereich im gleichen Ausmaß ausgereift abgeschlossen werden konnte. Rückblickend war es suboptimal, einzelne Oberflächen- und Datenflussentscheidungen zunächst pragmatisch statt konsequent entlang einer stabilen Struktur zu treffen.

8.3.3 Lernfortschritt und zentrale Erkenntnisse

Der wichtigste Erkenntnisgewinn meiner Arbeit liegt darin, dass Frontend-Entwicklung in einem solchen Projekt weit mehr ist als das Erstellen sichtbarer Seiten. Entscheidend ist, wie Daten aus dem Backend übernommen, sinnvoll strukturiert, sicher geschützt und für Nutzende verständlich dargestellt werden. Gerade bei der Arbeit mit Keycloak und bei der Umsetzung der Abo-Logik wurde deutlich, dass gutes Frontend-Design immer auch Architektur- und Integrationsarbeit ist.

Rückblickend war es richtig, den Fokus klar auf Filme und auf die zentrale Abo-Verwaltung zu legen. Diese Eingrenzung machte es möglich, die wesentlichen Funktionen tatsächlich umzusetzen und zugleich wichtige praktische Erfahrungen mit moderner Webentwicklung zu sammeln. Insgesamt sehe ich meinen Beitrag daher als gelungen an, auch wenn der erreichte Stand nicht als vollständig abgeschlossen, sondern als solide und fachlich nachvollziehbare Grundlage für weitere Ausbauschritte zu verstehen ist.

9 Ausblick

9.1 Mögliche Erweiterungen der Anwendung

Der aktuelle Projektstand zeigt, dass mit Mediahub eine funktionale Grundlage für die zentrale Verwaltung persönlicher Streaming-Abonnements geschaffen wurde. Gleichzeitig ergeben sich aus der bestehenden Lösung mehrere sinnvolle Erweiterungsmöglichkeiten, die den praktischen Nutzen der Anwendung in Zukunft weiter erhöhen könnten.

Eine naheliegende Weiterentwicklung betrifft die inhaltliche Erweiterung über Filme hinaus. Im Rahmen der Diplomarbeit wurde bewusst ein klarer Fokus auf Filme gelegt, um den Projektumfang beherrschbar zu halten und die Kernfunktionen sauber umzusetzen. Zukünftig könnte die Anwendung jedoch auch auf Serien, Mediatheken oder weitere audiovisuelle Inhalte ausgedehnt werden. Eine solche Erweiterung würde die Plattform deutlich breiter aufstellen, müsste aber fachlich und technisch sorgfältig geplant werden, damit die Benutzeroberfläche weiterhin übersichtlich bleibt.

Ein weiterer sinnvoller Ausbauschritt betrifft die Abo-Verwaltung selbst. Derzeit können zentrale Informationen wie Anbieter, Abrechnungszyklus und Datum der letzten Abbuchung erfasst und gespeichert werden. Darauf aufbauend wären Funktionen wie automatische Erinnerungen vor bevorstehenden Zahlungen, Hinweise auf Kündigungszeitpunkte oder einfache Kostenübersichten denkbar. Gerade solche Ergänzungen würden den ursprünglichen Problembereich der verlorenen Kostenübersicht noch direkter adressieren.

Auch die bereits begonnene Verlaufs- beziehungsweise History-Funktion könnte in Zukunft gezielter ausgebaut werden. Statt nur einen Teilaspekt abzudecken, wäre eine weiterführende Nutzung denkbar, etwa zur Dokumentation von angesehenen Inhalten, früheren Suchanfragen oder zuletzt betrachteten Filmen. Damit ließe sich der Nutzen der Anwendung stärker personalisieren.

9.1.1 Potenzial einer mobilen Anwendung

Ebenfalls denkbar wäre eine zukünftige Umsetzung als Android-Anwendung. Für die Nutzung auf Smartphone und Tablet könnte dies grundsätzlich sinnvoll sein, da Nutzende unterwegs schnell ihre gespeicherten Abonnements, Filmverfügbarkeiten oder bevorstehende Abbuchungen prüfen könnten. Gerade für eine Anwendung mit persönlichem Verwaltungscharakter wäre ein mobiler Zugriff daher praktisch. Gleichzeitig ist kritisch zu berücksichtigen, dass die bestehende Webanwendung bereits auf verschiedenen Geräten nutzbar ist und eine zusätzliche Android-Version einen eigenen Entwicklungs- und Wartungsaufwand verursachen würde. Aus heutiger Sicht wäre eine mobile App daher eine sinnvolle Option für die Zukunft, jedoch keine zwingend notwendige Voraussetzung für den praktischen Einsatz der Anwendung.

Aus Frontend-Sicht besteht außerdem Potenzial in einer weiteren Verfeinerung der Benutzerführung. Dazu zählen eine noch klarere Struktur einzelner Ansichten, zusätzliche Komfortfunktionen in Formularen, eine weiter verbesserte visuelle Rückmeldung bei Benutzeraktionen sowie eine noch stärkere Optimierung für unterschiedliche Bildschirmgrößen. Die bestehende Anwendung ist bereits funktional, könnte aber durch solche Verbesserungen in ihrer täglichen Bedienung noch reifer wirken.

9.2 Zukunftspotenzial der Plattform

Das Zukunftspotenzial von Mediahub liegt vor allem in der Verbindung von persönlicher Abo-Verwaltung, externer Filminformation und benutzerspezifischer Auswertung. Diese Kombination hebt die Anwendung von reinen Filmdatenbanken ebenso ab wie von einfachen Notiz- oder Verwaltungslisten für Abonnements. Die Plattform besitzt damit das Potenzial, sich zu einer zentralen Anlaufstelle für die persönliche Orientierung im fragmentierten Streaming-Markt weiterzuentwickeln.

Besonders relevant ist dabei, dass das Grundkonzept nicht nur für die aktuelle Problemstellung sinnvoll ist, sondern auch auf zukünftige Entwicklungen im Medienbereich reagieren kann. Je mehr Streaming-Anbieter, Plattformmodelle und hybride Angebote entstehen, desto größer wird der Bedarf an einer übersichtlichen, benutzerorientierten Verwaltungs- und Recherchelösung. Die in der Diplomarbeit entwickelte Grundlage zeigt, dass ein solcher Ansatz technisch und fachlich realisierbar ist.

Darüber hinaus bietet die im Projekt gewählte Architektur eine gute Basis für spätere Erweiterungen. Durch die Trennung von Frontend, Backend, externer Filmquelle und Authentifizierungssystem können einzelne Bereiche unabhängig weiterentwickelt werden. Das ist insbesondere für größere Ausbauschritte wichtig, da neue Funktionen nicht zwingend eine vollständige Neuentwicklung erfordern würden, sondern auf der bestehenden Struktur aufbauen können.

Insgesamt liegt das Zukunftspotenzial der Plattform daher weniger in einer einzelnen Zusatzfunktion als in der Möglichkeit, aus einer bereits funktionierenden Kernlösung schrittweise ein umfassenderes System für die Verwaltung und Nutzung digitaler Medienabonnements zu entwickeln. Die Diplomarbeit stellt dafür keinen Endpunkt dar, sondern einen belastbaren Ausgangspunkt.

10 Zusammenfassung

Im Rahmen der Diplomarbeit wurde mit Mediahub eine Webanwendung konzipiert und umgesetzt, die das Ziel verfolgt, persönliche Streaming-Abonnements zentral zu verwalten und mit Filminformationen zu verknüpfen. Ausgangspunkt war die Beobachtung, dass bei mehreren parallel genutzten Diensten leicht der Überblick über Kosten, Anbieter und tatsächliche Verfügbarkeiten verloren geht. Genau an diesem Problem setzt die Anwendung an, indem sie Verwaltungsfunktionen und filmbezogene Recherche in einer gemeinsamen Lösung zusammenführt.

Im technischen und fachlichen Mittelpunkt meines Projektteils stand dabei die Frontend-Entwicklung. Wesentliche Ergebnisse waren die Umsetzung einer Abo-Verwaltung, die Integration einer Filmsuche mit Detailansicht, die Darstellung passender Anbieterinformationen sowie die Einbindung von Keycloak für geschützte Bereiche. Dadurch konnte eine Benutzeroberfläche entwickelt werden, die nicht nur Informationen anzeigt, sondern einen konkreten praktischen Nutzen für die Zielgruppe erzeugt.

Gleichzeitig zeigte das Projekt, dass die Entwicklung einer solchen Anwendung nicht nur aus der Implementierung einzelner Funktionen besteht. Besonders die Verbindung zwischen Backend-Daten, Benutzeroberfläche, Authentifizierung und verständlicher Benutzerführung erwies sich als anspruchsvoll. Gerade aus diesen Herausforderungen ergaben sich jedoch wichtige Erkenntnisse für die eigene fachliche Weiterentwicklung, insbesondere im Umgang mit moderner Frontend-Architektur, Datenfluss und sicherheitsbezogenen Anforderungen.

Der erreichte Projektstand ist als funktionale und fachlich nachvollziehbare Grundlage zu bewerten. Nicht alle denkbaren Erweiterungen wurden vollständig umgesetzt, was vor allem auf die bewusste Fokussierung auf Filme und auf die Eingrenzung des Projektumfangs zurückzuführen ist. Diese Begrenzung war jedoch sinnvoll, da sie eine klare Zielsetzung und eine realistisch umsetzbare Lösung ermöglichte. Rückblickend zeigte sich zugleich, dass einzelne Entscheidungen in der Frontend-Struktur und bei der frühen Verteilung von Datenlogik suboptimal waren und heute konsequenter entlang klarer fachlicher Verantwortlichkeiten getroffen würden.

Zusammenfassend zeigt die Diplomarbeit, dass eine zentrale Verwaltung von Streaming-Abonnements in Verbindung mit Filminformationen sowohl technisch realisierbar als auch aus Sicht der Nutzenden sinnvoll ist. Mediahub stellt damit eine tragfähige Ausgangsbasis dar, auf der in Zukunft weitere Funktionen und fachliche Erweiterungen aufbauen können. Gleichzeitig bleibt der erreichte Stand bewusst als Zwischenstand zu bewerten: Die Verlaufsfunktion wurde nur teilweise umgesetzt und einzelne Komfortfunktionen der Abo-Verwaltung würden bei einer Weiterentwicklung gezielt ausgebaut werden.

Literaturverzeichnis

[1] Deloitte, „2025 Digital Media Trends," 2025, Zugriff am 03.04.2026. Online verfügbar: https://www2.deloitte.com/us/en/insights/industry/technology/digital-media-trends-consumption-habits-survey/2025.html

[2] The Movie Database, „Getting Started," Zugriff am 03.04.2026. Online verfügbar: https://developer.themoviedb.org/docs/getting-started

[3] ——, „Movie Watch Providers," Zugriff am 03.04.2026. Online verfügbar: https://developer.themoviedb.org/reference/movie-watch-providers

[4] Angular Team, „What is Angular?" Zugriff am 19.03.2026. Online verfügbar: https://angular.dev/overview

[5] ——, „Routing," Zugriff am 19.03.2026. Online verfügbar: https://angular.dev/guide/routing

[6] ——, „HTTP Client," Zugriff am 06.04.2026. Online verfügbar: https://angular.dev/guide/http

[7] ——, „Components," Zugriff am 06.04.2026. Online verfügbar: https://angular.dev/guide/components

[8] ——, „Forms," Zugriff am 19.03.2026. Online verfügbar: https://angular.dev/guide/forms

[9] MDN Web Docs, „Single-page application (SPA)," Zugriff am 19.03.2026. Online verfügbar: https://developer.mozilla.org/en-US/docs/Glossary/SPA

[10] A. Freeman, *Pro Angular*. Apress, 2019.

[11] Angular Team, „Angular CLI," Zugriff am 06.04.2026. Online verfügbar: https://angular.dev/tools/cli

[12] Keycloak Team, „Keycloak Documentation," Zugriff am 19.03.2026. Online verfügbar: https://www.keycloak.org/documentation

[13] ——, „Keycloak," Zugriff am 19.03.2026. Online verfügbar: https://www.keycloak.org/

[14] Tailwind Labs, „Tailwind CSS Documentation," Zugriff am 19.03.2026. Online verfügbar: https://tailwindcss.com/docs

[15] ——, „Tailwind CSS," Zugriff am 19.03.2026. Online verfügbar: https://tailwindcss.com/

[16] ——, „Styling with Utility Classes," Zugriff am 06.04.2026. Online verfügbar: https://tailwindcss.com/docs/styling-with-utility-classes

[17] ——, „Responsive Design," Zugriff am 06.04.2026. Online verfügbar: https://tailwindcss.com/docs/responsive-design

[18] ——, „Hover, Focus, and Other States," Zugriff am 06.04.2026. Online verfügbar: https://tailwindcss.com/docs/hover-focus-and-other-states

[19] ——, „Install Tailwind CSS with Angular," Zugriff am 06.04.2026. Online verfügbar: https://tailwindcss.com/docs/installation/framework-guides/angular

[20] GitHub, Inc., „Git Workflows," Zugriff am 19.03.2026. Online verfügbar: https://docs.github.com/en/get-started/getting-started-with-git/git-workflows

Abbildungsverzeichnis

1 Graphische Gesamtarchitektur der Anwendung 27
2 Anbieteranzeige innerhalb der Film-Detailansicht 30
3 Erstes Entity-Relationship-Diagramm der Anwendung 33
4 Überarbeitetes Entity-Relationship-Diagramm nach Integration von Keycloak 34
5 Erfassungsmaske für ein Streaming-Abonnement 49
6 Abo-Verwaltung mit Such-, Filter- und Bearbeitungsfunktionen 50
7 Film-Detailansicht mit Poster, Metadaten und Kurzbeschreibung 54
8 Film-Detailansicht mit Abgleich gespeicherter Abonnements und Anbieterübersicht 54
9 Geschützter Bereich der Abo-Verwaltung nach erfolgreicher Anmeldung 57